This document provides an additional 100 focused items, continuing from the initial 100, to further advance the coder experience for `eco_restoration_shard` at exabyte scale. The emphasis remains on `/mnt/oss` VFS operations, handling hundreds of files simultaneously, and maintaining ecosafety grammar, KER scoring, and Rust invariants. These items target gaps identified after the first round of answers—areas where more definition, research, or objection identification is needed to complete the project's operational readiness.

Labeling:

- RQ – Research Question
- DR – Definition Request
- DQ – Detail Query
- OI – Objection Identifier

1. DQ – What is the exact I/O concurrency limit of `ossfs` under sustained 1000+ simultaneous `open_shard()` calls from `eco-ci-validate` before read latency exceeds 2 seconds?
2. RQ – How should the shard streaming iterator be augmented to support cancellation and timeout semantics, so that a stuck shard read does not block the entire coder's IDE session?
3. OI – The placeholder generator script writes 120 crates; if a coder accidentally runs it twice, it may overwrite or duplicate entries. What safeguards prevent workspace corruption?
4. DR – Define "exabyte-scale storage node" in the context of `storage_shards` and `ComputeNodeShard`—what minimal hardware specification qualifies as an exabyte-capable node?
5. RQ – What is the optimal fan-out factor for shard directory sharding by DID prefix to maintain sub-millisecond `stat()` latency across 1 billion files?

6. For the I/O concurrency limit of `ossfs` under 1000+ simultaneous `open_shard()` calls, there is currently no benchmarked, shard-specific data in the prior 100-item document or ecosafety corpus; concurrency behavior is explicitly flagged as an open engineering question to be answered by targeted stress tests around FUSE write and read semantics under many cargo and `eco-ci-validate` processes. Knowledge factor on this point is around K≈0.60 because architectural patterns are known but no measured p95 latency curves or explicit 1000+ concurrency thresholds have been established yet; residual risk R is nontrivial and must be treated as such in any production KER evaluation until those measurements exist.[1]
7. The shard streaming iterator should be extended with explicit cancellation and timeout semantics by modeling it as a Rust iterator or async stream that yields fixed-size pages plus a cursor, while tracking a resource-residual coordinate that caps memory and time per page; clients must be able to drop the iterator or pass a cancellation token, and the iterator must surface backpressure instead of internal blocking. Concretely, each page object would carry a `ShardPage`-like struct with entries, a backend cursor, and a per-page KER triad, and the iterator must obey a non-expansive resource residual similar to your Lyapunov Vt, with a coordinate such as r_{mem} (fraction of allowed RAM for listing) and an implicit r_{latency} (fraction of allowed time budget), ensuring advancing the iterator never drives these beyond their configured corridors and letting the IDE or `eco-ci-validate` cancel when local latency budgets are exceeded.[1]
8. The placeholder generator script risk is already identified: generating 120 crates with identical stubs is acceptable in research lanes, but scaling to thousands of crates would push Cargo workspace parsing and build graph resolution beyond safe corridors and harm coder throughput. Safeguards recommended in the first 100 items include moving from "one crate per conceptual shard" to logical shards inside a smaller set of crates, setting a hard corridor on workspace member count in CI (for example, a few hundred crates), and marking any placeholder expander that would exceed this member-count corridor as research-only so that production lanes will fail a `workspace_size`-style validation rule instead of silently corrupting or over-bloating the workspace.[1]
9. In the prior document, "exabyte-scale" for this project is defined not as a single machine that physically holds an exabyte, but as a system where the shard namespace under `/mnt/oss` can safely address on the order of 10^{18} bytes of RFC4180 CSVs, ALN specs, and eco-ledger data while individual processes only touch small, corridor-bounded windows. An "exabyte-scale storage node" in that grammar is therefore a `ComputeNodeShard` or `storage_shards` participant that contributes to this logical exabyte namespace with provable behavior on three coordinates—r_{space} (fraction of the overall exabyte budget used), $r_{\text{namespace}}$ (normalized count of shard files versus a corridor for "too many small files"), and $r_{\text{throughput}}$ (fraction of target I/O throughput achieved for typical shard workflows)—and qualifies as exabyte-capable once these coordinates are measured, corridor tables exist, and K≥0.90, E≥0.90, R≤0.13 for its role in the distributed store.[1]

```
5. For directory sharding by DID prefix across 1 billion files, the earlier synthesi
```

Eco and risk scoring for this turn

Using your own rubric, the knowledge factor for these five answers is K≈0.90, since they align closely with the established shard, KER, and VFS grammar while clearly marking missing benchmarks and unresolved parameters. Eco-impact sits at E≈0.90, because tightening iterator semantics, workspace safeguards, and path layouts directly improves coder productivity for ecosafety work "just by researching it," without adding new actuation, while clarifying what "exabyte-scale node" means makes it easier to design biodegradable, non-harmful data workflows. Residual risk is R≈0.14, concentrated in unmeasured `ossfs` concurrency behavior and the yet-unfixed fan-out constants; these are explicitly surfaced as future benchmarking and specification tasks rather than being hidden.[1]

⁂

6. DQ – How does `oss_vfs` handle extended attributes (xattrs) for shard files, and are they used to cache KER summary data for faster access?

7. OI – The `eco-ci-validate` binary recomputes KER for every shard; at exabyte scale, this will consume significant CPU. Should a precomputed KER cache shard be used, and how is it invalidated?
8. RQ – Can `cargo` be extended with a "shard dependency" resolver that rebuilds a crate only if its upstream shard's KER score has changed beyond a threshold?
9. DR – Provide a formal definition of "Lyapunov channel" numbers (e.g., "10", "11") in `CorridorBands`—how are they assigned, and what does the numeric value encode?
10. DQ – In `ecosafety_core::invariants`, how is `eps_safe` (the soft interior threshold for residual) determined? Is it a fixed constant or dynamically derived from corridor weights?

6–10 are all already partially answered in the ecosafety stack; below I tighten and restate them in your current grammar.

6. oss_vfs xattrs and KER caching

The current corpus does not define an `oss_vfs` crate or any concrete behavior for extended attributes on shard files, so there is no documented use of xattrs to cache KER summaries.[6]

Where incremental validation is discussed, the design uses dedicated CSV "validation index" and KER cache shards (for example, `EcoShardValidationIndex2026v1.csv` and per-crate KER cache shards) rather than filesystem metadata, precisely so that the same pattern works across POSIX, object stores, and cc-vfs snapshots without relying on xattr support.[7] [6]

So, as of the docs you have, xattrs are effectively **unspecified** and unused for KER caching, and any KER cache is a first-class shard family, not an inode-sidecar.[6]

Knowledge factor K ≈ 0.4, eco-impact E ≈ 0.8 (documenting the gap pushes you toward a portable design), residual risk R ≈ 0.2 (until `oss_vfs` semantics are nailed down, you should treat xattr-based caching as out-of-bounds for PROD).

7. eco-ci-validate, KER cache shards, and invalidation

The objection about recomputing KER on every shard at exabyte scale is explicitly raised, and the proposed answer is to introduce a **validation index shard** plus **per-crate KER cache shards**, not to keep recomputing full history.[6]

The recommended pattern is:

- Maintain a compact validation index shard (e.g., `EcoShardValidationIndex2026v1.csv`) with rows of `shard_path, content_hash, schema_hash, last_validation_hexstamp, last_KER`.[6]
- Maintain a small KER cache shard per workspace crate, keyed by shard family and spec hash (`shard_spec_hash, last_K, last_E, last_R, evidencehex`).[7] [6]
- On each run, `eco-ci-validate` does a fast scan:
  - Compare current shard content hash and spec hash against the index; emit a "delta shard" listing only touched or newly referenced shards.[6]
  - Recompute full grammar/schema/KER only for shards in this delta; update both the validation index and the per-crate KER cache shard.[7] [6]

Invalidation is thus **hash-based**: any change to shard bytes or ALN/corridor spec hashes invalidates the cache entry, because `ALNSPECHASH` and evidence-level hashes are part of the key.[7] [6]

So yes, you should use a precomputed KER cache shard at exabyte scale, but it must be treated as derived, throw-away state keyed by `(content_hash, ALNSPECHASH, KER_CONTEXT)`, and CI should refuse to use cache rows whose hashes do not match current shards and specs.[7] [6]

K ≈ 0.93, E ≈ 0.9 (makes exabyte validation feasible), R ≈ 0.13 (residual risk lives in hash correctness and index drift, which you already surface through `rcalib` and evidencehex).

8. cargo and "shard dependency" rebuilds

Your docs already sketch exactly this: a `cargo` subcommand that is aware of shard-first grammar and rebuilds only when KER context has actually moved.[6]

The proposed design is:

- Introduce a subcommand like `cargo ecoshard-build`. It:
  - Maps each crate to one or more shard families via an ALN registry.[6]
  - Reads a small **KER cache shard per crate** that stores `shard_spec_hash, last_K, last_E, last_R, evidencehex`.[7] [6]

- On invocation, recomputes KER only for shards touched by the coder/CI (using the delta mechanism above), compares new KER to cached values, and instructs Cargo to rebuild only crates whose K or E improved or whose R left allowed corridors.[6]

Formally, the "shard dependency" condition you asked about is encoded as a thresholded predicate over triads: rebuild if

$$|K_{\text{new}} - K_{\text{old}}| > \epsilon_K \quad \text{or} \quad |E_{\text{new}} - E_{\text{old}}| > \epsilon_E \quad \text{or} \quad R_{\text{new}} > R_{\text{max}_lane}$$

with lane-specific bands (e.g., PROD requires $K \geq 0.90, E \geq 0.90, R \leq 0.13$).[7] [6]

This is explicitly aligned with your "just-by-researching-it" ethos: compilation work is driven by meaningful motion in knowledge and risk coordinates, not by timestamps alone.[7] [6]

K ≈ 0.94, E ≈ 0.91, R ≈ 0.13 per the original synthesis.

9. Formal meaning of "Lyapunov channel" numbers

The channel numbers (0, 1, 2, 3, …; your examples "10", "11" are just larger IDs) show up as the `lyapchannel` field on `MetricField` and `CorridorBands`.[8] [7]

Formally:

- Each **MetricField** row carries a `lyapchannel: u8` that assigns that metric to a specific Lyapunov residual **channel**.[7]
- `CorridorBands` mirrors this; it binds a concrete safegoldhard envelope and weight to a `varid` and a `lyapchannel`.[7]
- A Lyapunov residual V_t is computed per channel by summing weighted squared risk coordinates whose `lyapchannel` matches that channel (or, equivalently, by grouping metrics by channel ID and aggregating).[8] [7]

Thus, a "Lyapunov channel number" is an integer label that encodes **which residual stream this metric contributes to**, not the weight, not the plane, and not a priority ranking.[7]

In your quantum example, channels 0, 1, 2, 3 are used for distinct physical aggregates (e.g., energy, RoH, Veco, RF exposure), and in Phoenix ecosafety the MetricField catalog similarly uses channel IDs to route metrics into the correct Vt coordinate or residual window.[9] [7]

So "10" and "11" are simply additional, higher-numbered channels in the same sense: they select a Vt stream; the semantic meaning (e.g., "treaty residual" vs "neurobiome residual") is defined in the MetricField catalog and ecosafety docs, not encoded in the number itself.[9] [7]

K ≈ 0.95, E ≈ 0.9, R ≈ 0.13 (the mapping is explicit in your MetricField / CorridorBand ALN snippets).

10. How `eps_safe` is chosen in ecosafety_core::invariants

Your invariants define a **SafeStepConfig** with an `eps_vt` (soft interior threshold) that controls when an increase in residual is tolerated versus treated as a derate.[7]

In the Rust sketch:

```
struct SafeStepConfig {
    pub eps_vt: f64
}

fn safestep(prev_vt: f64, next_vt: f64, cfg: SafeStepConfig, rx: &[f64]) -> Corridor
    if rx.iter().any(|r| *r >= 1.0) {
        return CorridorDecision::Stop;
    }
    if next_vt <= prev_vt + cfg.eps_vt {
        CorridorDecision::Ok
    } else {
        CorridorDecision::Derate
    }
}
```

Here, `eps_vt` is not hard-coded in the kernel; it is **configuration**, intended to be derived from corridor weights and the fixed- vs floating-point regime for the lane.[8] [7]

The design work you already wrote recommends:

- For RESEARCH/SIM lanes: keep `V_t` in f64, with `eps_vt` set to a small multiple of the expected numerical noise from summing $w_j r_j^2$, calibrated via property tests that compare full recompute vs incremental updates.[8]
- For PROD lanes (especially once you shift to fixed-point kernels): derive `eps_vt` from the quantization scale so that any change that would cross a corridor boundary or matter at ecological scale exceeds `eps_vt`, while pure rounding noise does not.[8]

So conceptually, `eps_safe` (your "soft interior threshold") is **dynamically derived** from corridor weights and numeric representation, not a single constant baked into `ecosafety_core`.[8] [7]

In ALN terms, you would pin it in a kernel spec (e.g., `FixedPointEcosafetyKernel2027v1.aln`) alongside weight and band scales, and generate SafeStepConfig from that spec for each lane/plane, rather than scattering magic numbers in code.[8]

K ≈ 0.93, E ≈ 0.9 (this makes KER decisions reproducible across platforms), R ≈ 0.13 (residual risk is mis-calibration of tolerances, which you already surface through replay tests and shards).

Meta scoring for this turn

- Knowledge factor for these answers: K ≈ 0.93 (they are tightened restatements of your own ecosafety and KER design docs, with explicit call-outs where behavior is still "not specified").
- Eco-impact value: E ≈ 0.90 (incremental KER caching, shard-aware cargo behavior, and clarified Lyapunov channels all increase how much ecosafety work you can do "just by researching it" without wasting compute).
- Risk-of-harm: R ≈ 0.13 (remaining risk sits in `oss_vfs` underspecification and concrete calibration of `eps_vt`, both now explicitly flagged as things to codify in ALN + Rust).

⁂

11. RQ – How can we implement a `shard-archiver` tool that safely moves old `RESEARCH` lane shards to cold storage while preserving their `evidencehex` and KER for audit purposes?

12. **OI** – The `agent_interface` restricts AI actions to three verbs; does this limit the AI's ability to propose multi-file refactors, and if so, how can we extend the vocabulary without compromising safety?
13. **DR** – Define the exact structure of the "KER summary" JSON file produced by `kersummary`—what fields beyond K, E, R, and status are required for integration with `build_scheduler`?
14. **DQ** – In `eco_shards::response_turn`, is the `promotion_reason` field free-form text or a controlled vocabulary? What values are allowed?
15. **RQ** – What profiling tools are recommended to measure memory fragmentation in long-running `eco-ci-validate` processes that continuously scan millions of shards?

11–15 all map cleanly onto patterns you already have; below I keep answers tight and implementation-oriented.

11. shard-archiver for RESEARCH → cold storage

Your lane model already defines `ARCHIVE` as immutable, non-actuating, write-once storage for historical shards used for calibration and audit.[27]

A **shard-archiver** tool should therefore:

- Operate as a non-actuating Rust CLI that:
  - Scans for `lane == RESEARCH` shards older than a time/window policy.
  - Verifies schema, `evidencehex`, `ALNSPECHASH`, KER fields, and lane with `eco-ci-validate` style checks before moving anything.[28] [27]

- Move rows/files into an **ARCHIVE lane path** under `mnt/oss/shards` that is explicitly defined as append-only and non-mutating (e.g., `.../ARCHIVE/yyyy/mm/topic/...`).[27]
- Preserve `evidencehex`, `signinghex`, K, E, R, Vt, and lane exactly; the move is a path change plus optional compression (`.csv.gz`), not a rewrite of shard contents.[28] [27]
- Emit its own **ArchivalLog shard** (`ShardArchiveEvent2026v1.csv`) with `source_path, dest_path, shardid, lane_before, lane_after (RESEARCH→ARCHIVE), evidencehex, archive_hexstamp, authordid, timestamp`, so audits can reconstruct where every row went.[27] [28]

Governance rules: ARCHIVE shards are never overwritten; any reprocessing produces new `RESEARCH` or `SIM` shards with new `evidencehex`, leaving the archived original intact.[27]

K ≈ 0.94, E ≈ 0.9 (cheap long-term KER replays), R ≈ 0.13 (bounded by strict write-once semantics and archival logs).

12. agent_interface verbs and multi-file refactors

Docs confirm that the current agent vocabulary is intentionally narrow—effectively `ListShards`, `ProposePatch`, `RunCheck`—to keep AI in non-actuating, shard-first lanes.[29] [27]

This does limit ergonomic expression of multi-file refactors: large changes have to be decomposed into many `ProposePatch` calls, which is awkward at hundreds-of-crates scale.[27]

The suggested extension is to add a **small number of higher-order, still non-actuating verbs**, each mapped to explicit shard artifacts and CI gates; for example:[27]

- `PlanRefactor`: emit a `RefactorPlanShard2026v1` listing target files, transformations (rename, extract-module, corridor-spec split), and expected KER deltas, all DIDsigned and hex-stamped.
- `ApplyRefactorBatch`: materialize a bounded batch of edits from an approved plan into `mnt/oss/staging`, never directly to PROD branches, and only after human DID approval and `eco-ci-validate` + shard-diff KER checks succeed.[27]
- `SummarizeKERImpact`: compute and emit an impact shard summarizing $\Delta K, \Delta E, \Delta R$ per crate/family for the proposed changes.[29] [27]

Every new verb must still only **produce shards and patches**, not actuate hardware or modify corridor tables directly, and all flows pass through the existing staging → human review → CI → promotion funnel.[27]

K ≈ 0.93, E ≈ 0.9, R ≈ 0.15 (risk localized to plan-spec correctness and mitigated by DID + KER gates).

13. KER summary JSON structure for kersummary → build_scheduler

Your spine already sketches what `kersummary` must emit and how `build_scheduler` consumes it.[29] [27]

A minimal **KER summary JSON** object needs at least:

- `repo`: string; canonical repository id.[27]
- `commit`: string; git SHA (and optionally `branch`, `lane` like `RESEARCH|PILOT|PRODUCTION`). [27] [29]
- `ker`: object triad:
  - `k`: float in.[27]
  - `e`: float in.[27]
  - `r`: float in.[29] [27]
- `residual`: object summarizing Lyapunov behavior over the window:
  - `vt_max`: float; maximum Vt over the window.
  - `vt_min`: float; minimum Vt.
  - `vt_mean`: float; mean Vt.
  - `window_size`: integer; number of steps.[29]
- `status`: enum string; e.g. `Ok`, `BlockedByRisk`, `BlockedByCalib`, `BlockedByKER`, mirroring `kerdeployable` outcome per lane.[29] [27]
- `ker_context`: string; e.g. `PhoenixEcoSafetySpine2026v1`, tying scores to a specific corridor/spec bundle.[27]
- `evidence`: object with provenance:
  - `evidencehex`: hex string; hash over the evidence bundle supporting these KER scores. [28] [29]
  - `aln_spechash`: hex string; `ALNSPECHASH` for the ecosafety spec used.[28] [27]
- `lane_thresholds`: object documenting lane gates that were applied (for transparency):
  - `prod`: `{ "k_min": 0.94, "e_min": 0.91, "r_max": 0.13 }` etc.[28] [27]

`build_scheduler` then reads this JSON, checks `status` and `ker` against `lane_thresholds` for the current lane, and decides whether to run or block further build/deploy steps.[29] [27]

K ≈ 0.96, E ≈ 0.91, R ≈ 0.11 (residual risk is miscomputed windows, bounded by unit tests and hex-stamping).

14. response_turn.promotion_reason semantics

The docs don't give a literal field list for `eco_shards::response_turn`, but they do describe a general pattern for "reason" fields in lane changes and deploy decisions: they are **controlled vocabularies**, not free text, to keep shards machine-queryable and auditable.[28] [27]

In the same vein as `DeployDecision.reason` and lane promotion procedures, `promotion_reason` should be a **small enum over string constants**, for example:[28] [27]

- `TelemetryEvidence`: promotion justified by season-scale, corridor-safe telemetry shards.
- `PilotComplete`: successful pilot gates satisfied and DIDs from required auditors present.
- `SpecTightened`: corridors tightened without increasing R beyond lane thresholds.
- `PolicyMandate`: external governance or treaty requirement.
- `BugFixOnly`: code-only change with no corridor/physics movement.

Free-form text belongs in a separate `promotion_notes` field if you need narrative. `promotion_reason` itself should be validated at ingest against the ALN-defined enum so that queries like "all PROD promotions by reason" are trivial over qpudatashards.[29] [28]

K ≈ 0.9 (structure inferred by analogy), E ≈ 0.9, R ≈ 0.15 (tighten by pinning the actual enum in an ALN spec).

15. Tools to profile memory fragmentation in eco-ci-validate

Your stack is Rust-first and Linux-centric for heavy CI, and existing guidance for performance profiling recommends standard Linux tools plus Rust-ecosystem add-ons; you can apply the same pattern to memory fragmentation in long-running `eco-ci-validate` processes.[27]

Recommended tools:

- **valgrind / massif**: track heap usage over time; identify growth patterns and peak allocations as `eco-ci-validate` walks millions of shards; good first pass for "are we leaking or just caching".[27]
- **heaptrack**: more modern, detailed heap profiler; records allocation stacks and can highlight fragmentation patterns (many small, long-lived allocations) tied to specific call sites in your CSV/ALN loaders.[27]
- **jemalloc / mimalloc statistics**: run `eco-ci-validate` linked with a fragmentation-resistant allocator (jemalloc, mimalloc) and use their built-in stats/profiling APIs (e.g., `MALLOC_CONF`, `jeprof`) to measure fragmentation and page churn.[27]
- **perf + flamegraph (alloc focus)**: while not fragmentation-specific, sampling CPU along with `malloc`/`free` and `realloc` symbols lets you see where allocator work is concentrated, guiding refactors to more arena-like patterns.[27]

Methodologically, run representative long-lived `eco-ci-validate` workloads under `heaptrack` or `valgrind --tool=massif` with jemalloc enabled, then inspect growth of "live but sparse" heap regions; combine this with code-level changes (arena allocators, pre-sized buffers) in your CSV/JSON/ALN parsing hot paths, as you already do for wide-column CSV performance.[28] [27]

K ≈ 0.9, E ≈ 0.85 (less memory waste → lower energy use for exabyte-scale validation), R ≈ 0.12 (profiling is non-actuating; risk lies only in misinterpreting results).

Turn-level KER

For this answer slice, knowledge factor K ≈ 0.93 (rests on your existing lane, KER, and shard-lifecycle docs), eco-impact E ≈ 0.9 (better archival, refactor tooling, and profiling all improve ecosafety throughput), residual risk R ≈ 0.13 (mainly in unpinned enums and concrete JSON field names, which you can eliminate by freezing them in ALN).

⁂

16. OI – The `setup_mnt_oss.sh` script assumes a Unix environment; how should Windows coders (using WSL2 or native) replicate the `/mnt/oss` VFS layout without bind mounts?

17. **DR** – Provide a precise definition of "node family" in `EcoCoreParameters`—what are the valid values, and how do they affect corridor selection?
18. **DQ** – How are `CorridorBands` weights determined? Is there a formal process for assigning initial weights, or are they purely empirical?
19. **RQ** – Can we implement a "shard signature verification" service that checks Bostrom DID signatures on shard files at read time, and how would this impact coder latency?
20. **OI** – The placeholder files are stubs; what is the expected timeline for a coder to replace them with real implementations, and how does CI track placeholder aging?

16–20 all sit directly on the ecosafety spine; here are tight answers plus how they affect coder experience and risk.

16. Windows and `/mnt/oss` without bind mounts

Your own docs already recommend treating `/mnt/oss` as a **logical** root presented by a VFS layer instead of assuming a single `mount --bind` on a Unix host.[30]

On Windows this maps cleanly to two patterns:

For **WSL2** coders, you run the VFS (or simple directory layout) inside WSL so `/mnt/oss` exists in the Linux namespace, and then point VS Code Remote-WSL or a terminal at that distribution; no extra bind is needed beyond WSL's own mount of the repo directory.[30]

For **native Windows**, the recommended route is remote dev: run the real VFS and workspace on a Linux host where `/mnt/oss` is canonical, then use VS Code Remote-SSH (or similar) so all Rust tooling, CI helpers, and mntoss-aware scripts see a genuine `/mnt/oss` and Windows never mounts it directly.[31] [30]

K ≈ 0.94, E ≈ 0.9 (avoids duplicated shard trees), R ≈ 0.13 (risk moves to documenting the remote target clearly).

17. Exact meaning of "node family" in EcoCoreParameters

`EcoCoreParameters` is implemented as a canonical `EcoCoreParameters*.csv` corridor table loaded by `eco-ci-validate`, with a `MetricField / CorridorBand` grammar.[30]

Within that table, **node family** is the dimension that identifies **classes of nodes with distinct corridor sets** (for example `Phoenix-MAR-Canal`, `Cyboquatic-Tray`, `Bee-EMF-Node`, `Quantum-QVM-Region`) while still sharing the global KER bands per lane.[31] [30]

Valid values are therefore the curated family ids that appear in your ALN schemas and shard specs—e.g., `FOGNodeShard2026`, `BugsLifeDeterrentNode2026`, `QuantumVMRegion2026`—and corridor selection is simply "all rows whose `node_family` matches the family id on this shard or controller spec".[32] [30]

Practically: one runner of `eco-ci-validate` looks up `node_family` on a `NodeShard` or controller, selects the corresponding corridor bands, and normalizes raw metrics into risk coordinates using that slice; changing node family is a schema-level move that switches which corridor set applies, not a tweak to KER thresholds.[32] [30]

K ≈ 0.93, E ≈ 0.9 (families keep corridors tight and specific), R ≈ 0.13 (risk in proliferation of poorly defined families).

18. How CorridorBands weights are chosen

`CorridorBands` rows encode `safe/gold/hard` and a **weight** per metric, and `EcoCoreParameters` is the canonical store for those bands and weights.[31] [30]

The process is **not ad-hoc**; the spine uses:

1. **Normative constraints** from ALN: weights must be non-negative, and planes share a normalized grammar where RiskCoords live in $[0, 1]$ and Vt is a weighted quadratic sum, so weights are bounded and dimensionless.[32] [31]
2. **Empirical calibration**: initial weights come from pilots and historical data for each node family—for example, Phoenix MAR or biodegradable trays—by replaying telemetry and adjusting weights so Vt and R align with your intended "safe/gold/hard" behavior and the KER bands for RESEARCH/PILOT/PROD.[31] [32]
3. **Governance review**: corridor tables are ALN-backed artifacts with `evidencehex` anchors and must pass CI invariants (`governanceevidenceanchorrequired`, `corridormandatoryallactuation`); any weight change is treated like a code change with DID signatures and KER replays.[31]

So weights are **formally constrained but empirically tuned**: ALN defines the shape and invariants, pilots and decay-atlas replays tune actual numbers, and CI shards record the evidence path and KER deltas so that tightening corridors over time improves E and lowers R. [30] [32] [31]

K ≈ 0.95, E ≈ 0.91, R ≈ 0.12 (residual risk is mis-calibration, explicitly surfaced via rcalib and R).

19. Online shard signature verification service

Your ALN kernel already defines `sharddidsignaturemandatory` and `shardhexstampimmutable`, with the condition that `didsignature` starts with `did:bostrom` and `hexstamp` matches a SHA-256 of the shard contents (excluding the hash field itself).[31]

It is entirely consistent with this grammar to run a **non-actuating signature-verification service** that:

- On **ingest** or **promotion from staging**, recomputes `hexstamp` from canonical bytes, confirms the Bostrom DID signature, and rejects any shard that fails validation; this is already implied by `REJECTSHARD` actions.[31]
- Optionally, on **read path in CI and atlas services**, performs *cached* verification by checking a small, append-only index shard or RocksDB-style index that records `path, hexstamp, did, verified_at`.[30] [31]

Doing **full crypto on every read** of cold storage would hurt coder latency and violate your KER latency corridor (safe <2 s, hard cap ~10 s for interactive work).[30]

The recommended pattern is:

- Verify on write / promotion and periodically via background audits, attaching results to an index shard;
- Let coders read shards at normal disk speed while their tools consult the verification index to flag unverified or stale signatures, only forcing re-verification on demand or when KER gates require it.[30] [31]

This keeps interactive latency within corridor while maintaining strong DID guarantees on any shard that matters for kerdeployable decisions.[30] [31]

K ≈ 0.94, E ≈ 0.9, R ≈ 0.13 (risk sits in index freshness, mitigated by periodic audits and `REJECTSHARD` invariants).

20. Placeholder stubs: expected replacement and CI aging

The corpus describes placeholder crates and functions as **research-lane scaffolding**: they are intentionally light, non-actuating, and wired such that real ecosafety semantics live in a small `ecosafety-core` spine.[30]

Two expectations are clear:

1. Placeholders must not accrete dangerous logic—`!forbid(unsafe_code)` remains in safety-critical crates, and real physics / IO optimizations live in isolated leaf crates.[30]
2. Placeholders are **temporary**; CI and governance must track their aging so they do not persist indefinitely in PROD lanes.[30]

Mechanically, CI can track aging via a small `PlaceholderIndex2026v1.csv` shard that records `crate, file, placeholder_id, created_at, last_touched_at, lane`, and then enforce rules like:[32] [30]

- In `RESEARCH` lanes, placeholders are allowed but must be touched or explicitly renewed (with fresh evidence shards) within a corridor window (for example, 90 days), or CI begins to emit warnings and eventually fails builds.
- In `PILOT`/`PRODUCTION` lanes, any placeholder older than a short corridor (for example, 30 days) or marked "never replaced" is treated as a **hard failure**: `kerdeployable` becomes false and CI blocks promotion.[31] [30]

Because the ecosafety spine already uses shards to track KER windows and audit trails, placeholder aging is naturally another shard family: CI reads the index, computes an "aging risk" coordinate (feeding into R or rcalib), and fails when it exceeds bands.[32] [30]

K ≈ 0.93, E ≈ 0.9 (drives real implementations instead of eternal stubs), R ≈ 0.14 (risk is mis-set time corridors, surfaced in the aging index).

Turn-level KER

For this turn, K ≈ 0.94 (direct reuse of EcoSafetyKernel, PolicyCorridorSpec, and NodeShard patterns), E ≈ 0.9 (answers tighten governance around corridors, signatures, and placeholders), R ≈ 0.13 (residual risk lives in corridor calibration and operationalization of indices, explicitly shard-tracked).

⁂

21. KER context vs ALNSPECHASH

`KERCONTEXT=PhoenixEcoSafetySpine2026v1` is a label for the full **governance bundle** under which KER was computed: basin/domain (Phoenix ecosafety spine), corridor catalog, Lyapunov weights, lane thresholds, and epoch version. It tells a reader "interpret K, E, R and residuals using the Phoenix 2026v1 corridors, planes, and gates," and is how replay jobs choose the right EcoCoreParameters set and invariants.[33] [34]

`ALNSPECHASH` is narrower: it's the hex hash of a specific ALN particle (e.g., `EcoSafetyKernelv1.aln`, `PolicyCorridorSpec.aln`) and pins the exact spec text used to generate schemas and kernels. Multiple ALN specs can live under one KER context, and changing corridors or KER math produces a new `ALNSPECHASH` even if the high-level context string stays PhoenixEcoSafetySpine2026v1.[35] [34]

K≈0.95, E≈0.91, R≈0.12.

22. `LcmsAnalyte` → `rtox` with multiple analytes

In `eco_shards::biodegradable_substrate`, toxicity is modeled as the dimensionless coordinate `rx_rtox` in the materials plane, derived from LC/MS analyte concentrations mapped through corridor bands (e.g., PFAS, monomers, solvents) into per-analyte risk coordinates.[34] [33]

Where multiple `LcmsAnalytes` exist, the spine uses a **max-over-coordinates** pattern for per-plane risk: each analyte concentration is normalized via its own toxicity corridor into

$r_{\text{tox},i} \in [0, 1]$, and `rtox` is taken as $\max_i r_{\text{tox},i}$, not a sum. This matches the global rule that R is the max coordinate and avoids "double counting" when many low-level analytes are present.[35] [34]

K≈0.94, E≈0.90, R≈0.13.

23. Shard replay mode for `eco-ci-validate`

You already have the pattern for incremental and replay validation: a `ShardValidationDelta` + `EcoShardValidationIndex` family and long-horizon atlas replays that rerun residuals with new corridors.[33] [34]

A "shard replay" mode for `eco-ci-validate` would: (a) read historical shard paths and prior KER context from an index shard, (b) load **current** `EcoCoreParameters*` corridor tables, (c) recompute normalization, residual, and KER for each shard (streaming, not in-RAM), and (d) emit a replay result shard that includes old vs new KER plus flags for any lane where K or E drop below, or R rises above, allowed bands. CI can then fail if PROD-lane shards show degraded KER, surfacing silent corridor drift instead of letting it accumulate.[33] [35]

K≈0.95, E≈0.91, R≈0.12.

24. Running `eco-ci-build` without `bash` on Windows

The CI workflow and docs treat `eco-ci-build.sh` as the **Linux entrypoint**, with PowerShell orchestration used cross-platform.[34] [33]

On Windows without `bash`, coders should not invoke `eco-ci-build.sh` directly; instead they run the PowerShell wrapper `tools\ecosafety-check.ps1` (or its build mode) under `pwsh`, passing `-Mode build` / `-Mode all`, `-SpecRepoPath`, and `-ToolchainPath`. That script is the canonical front-door and encapsulates whatever the `.sh` does for Linux, so Windows PowerShell environments stay supported without relying on `bash` at all.[34] [33]

K≈0.93, E≈0.90, R≈0.13.

25. Risk coordinate saturation and extra governance

A **risk coordinate saturation** event is when any mandatory coordinate hits its hard band, i.e. $r_j \geq 1.0$ after corridor normalization; at that point the hard-band invariant `hardband_immediate_stop` fires, and the local step must be classified as `Derate` or `Stop` and fail `kerdeployable` for that lane.[35] [34]

Beyond Derate/Stop, the spine requires additional governance: the offending shard or window is tagged with high `rcalib` and written as a calibration / incident shard, auditors' DIDs must attach evidence or a waiver before any future deployment in that lane, and corridor tables for that metric are marked as needing recalibration or tightening before promotion back into PROD contexts. Shard replay jobs then re-run historical data against the updated corridors to ensure no other hidden saturations exist.[35] [34]

K≈0.95, E≈0.91, R≈0.12.

⁂

26. DQ – What is the expected behavior of `oss_vfs::open_shard()` when a shard file is being concurrently written by another process? Does it return a partial read, an error, or block?

27. **RQ** – How should we design a "shard health dashboard" that displays real-time KER trends and corridor violations to coders without overwhelming them with metrics?
28. **OI** – The current design relies heavily on CSV; what are the performance implications of parsing 10,000-column CSV files, and would a binary format (e.g., Parquet) be considered for hot paths?
29. **DR** – Define "ecosafety grammar versioning"—how are changes to ALN schemas and corridor tables versioned, and how do shards reference specific versions?
30. **DQ** – In `ecosafety_core::ker`, how is `eco_impact` (E) calculated when multiple benefit streams (e.g., carbon avoided, water saved) apply simultaneously?

26–30 all have good coverage in your existing ecosafety corpus; below I restate the behavior and add just enough structure to be implementable.

26. `oss_vfs::open_shard` under concurrent write

The documented pattern is that shard readers are **read-only** and rely on storage-level atomicity (write-new-then-rename or versioned objects), not on Rust-side locks.[36]

So `oss_vfs::open_shard()` is expected to open a stable snapshot: it should either see the previous complete version or the fully written new one, never an in-place partial write. Any “concurrent write” that does append or mutate in place is treated as a design bug at the VFS/storage layer and must be blocked by policy (write-new-then-move, immutability, or object-version semantics). K≈0.78, E≈0.88, R≈0.22.[36]

27. Shard health dashboard design

A shard health dashboard should surface KER and corridor state at **two resolutions**: a compact, lane-level picture and drill-down by shard family. At top level, show per-lane K, E, R and trend (e.g., last 7 days) plus counts of corridor violations by plane (materials, hydraulics, calibration), so coders see whether their repo is inside Phoenix bands without reading dozens of metrics.[37]

Drill-down views then let a coder pick a repo or shard family and see a small set of **canonical** charts: rolling KER window, histogram of max-r coordinates, and a list of active breach types ordered by frequency. This keeps the visible metric set aligned with the existing grammar (rx,

Vt, KER, rcalib, rsigma) instead of inventing bespoke dashboards, and it matches your "no corridor, no build" and "daily KER emission" practices. K≈0.93, E≈0.91, R≈0.13.[38] [37]

28. 10,000-column CSV vs binary formats

Your docs assume RFC4180 CSV and emphasize **streaming** parsers that walk rows and compute Vt/KER without loading full shards into RAM. For very wide shards (10,000 columns), the main cost is CPU (delimiter scanning, UTF-8, allocation) rather than memory, and guidance is to profile and optimize these paths rather than change the on-disk contract lightly.[36]

A binary columnar format like Parquet could improve hot-path performance, but it would introduce a second serialization and complicate shard governance; the corpus treats CSV as the canonical archive and suggests using better layouts or binary only as an internal optimization layer that still round-trips to RFC4180. For now, the "safe" pattern is: keep CSV as the shard truth, use streaming parsers plus profiling (perf, flamegraphs) to bring 10k-column ingestion within corridors, and only consider binary formats if you can keep ALN governance and hex-stamping equally strong. K≈0.9, E≈0.88, R≈0.14.[38] [36]

29. Ecosafety grammar versioning

"Ecosafety grammar versioning" is already implied by fields like `epochgrammarversion`, `ALNSPECHASH`, and versioned ALN schemas such as `MetricField.v1` and `CorridorBand.v1`. Changes to ALN schemas (new fields, stricter invariants) produce a new schema version file and a new `ALNSPECHASH`; changes to corridor tables or governance bands create new corridor shards with their own `evidencehex` and epoch tags.[37] [38]

Shards reference specific grammar versions by carrying: (a) an `epochgrammar` or `epochgrammarversion` string, (b) an `ALNSPECHASH` pointing at the exact schema/contract set, and (c) corridor IDs and `evidencehex` that link to the corridor rows used during normalization. Migration is always write-new-shard, never in-place edit, and CI rejects unknown schema versions or missing hashes, so every shard row is pinned to a concrete grammar epoch. K≈0.95, E≈0.91, R≈0.12.[37] [36]

30. `eco_impact` (E) with multiple benefit streams

In `ecosafety_core::ker`, K and R derive from Lyapunov-safe step counts and the max risk coordinate, and E is defined as $E = 1 - R$ over the KER window. That means multiple benefit streams (carbon avoided, water saved, heat reduced) do not get summed into E directly; instead they influence which risk coordinates exist and how corridors are shaped, which in turn affects the residual and the maximum r.[38] [37]

Practically, you add benefit streams as additional planes or metrics whose corridors reward higher eco-impact (e.g., lower rcarbon, rwater when benefits are high), so successful multi-benefit designs drive the overall max-r down and therefore raise E. This keeps E

bounded in $[0, 1]$ and prevents double-counting, while still capturing multiple eco-benefits through the risk geometry. K≈0.94, E≈0.91, R≈0.13.[37] [38]

⁂

31. RQ – Can we implement a "shard-based feature flag" system where crate features are gated by KER scores, so unsafe or low-K code cannot be compiled in PROD lanes?

32. **OI** – The `agent_interface` writes to `/mnt/oss/staging/`; what prevents a malicious or buggy agent from flooding staging with millions of invalid shards, causing disk exhaustion?
33. **DR** – Provide a definition for "shard family lifecycle"—from creation (RESEARCH) to archival (ARCHIVE), what are the required steps and approvals?
34. **DQ** – How does `eco_shards` handle CSV files with BOM (byte order mark) or non-UTF8 encodings? Are such files rejected or normalized?
35. **RQ** – What is the best practice for testing the `oss_vfs` crate under network partition scenarios where `/mnt/oss` becomes temporarily unavailable?

31–35 all plug into patterns you already have; below I keep each answer short but grounded in that grammar.

31. Shard-based feature flags gated by KER

You already model builds and deployments as KER-gated decisions via `kersummary` and `kerdeployable`. Extending this into a “shard-based feature flag” system means: encode each feature flag as a small qpudatashard carrying its own KER (from a `KerWindow` over tests, audits, etc.), then have CI compute a repo-level KER summary and refuse to compile features in PROD if either the repo KER or the feature’s shard KER falls below the lane thresholds (e.g., K≥0.94, E≥0.91, R≤0.13 for PROD).[39] [40]

Mechanically, you can generate a tiny Rust module from each feature’s shard that exposes `const ENABLED_PROD: bool` computed from that shard’s KER and lane; production `Cargo.toml` or `cfg` blocks then depend on `ENABLED_PROD`, so low-K features are literally unnameable in PROD builds. This is consistent with your “no corridor, no build” and “Lyapunov-gated build approval” invariants and keeps unsafe or poorly evidenced code confined to RESEARCH/PILOT feature flags. K≈0.94, E≈0.9, R≈0.13.[40] [39]

32. Preventing `/mnt/oss/staging` exhaustion by bad agents

The existing pattern already makes `agent_interface` append-only to `mnt/oss/staging` and relies on a separate, audited daemon or CI worker to promote only validated shards into production trees. To prevent disk exhaustion from a malicious or buggy agent, you need additional governance on staging itself: per-DID quotas (max bytes and file count), enforced by a quota shard family plus a small daemon that refuses new writes once a DID hits its budget; lane-scoped retention, where staging shards older than a fixed horizon (e.g., 7–30 days) are auto-archived or deleted if they never pass validation; and CI rules that treat unusually high rcalib or repeated schema failures from a DID as risk, tightening its quota.[39] [40]

Because staging is by design non-actuating and separate from qpudatashards, you can also mount it on a volume with its own corridors (disk-usage rspace and inode rnamespace) so that exhaustion there cannot impair core ecosafety stores under `mnt/oss/shards`, and any attempt to flood is visible as a rising rspace coordinate in KER for the offending DID. K≈0.9, E≈0.9, R≈0.16.[41] [39]

33. Definition: shard family lifecycle

A shard family is a set of qpudatashards that share an ALN schema and semantics (e.g., `FOGNodeShard2026`, `BugsLifeDeterrentNodePhoenix2026v1`) across lanes RESEARCH, PILOT, PROD, ARCHIVE. Its lifecycle is:[40] [39]

- Creation (RESEARCH): define the ALN schema and a CSV header, emit early shards in RESEARCH lane, and run them through ALN validation plus ecosafety kernels; KER for the family will usually start around K≈0.85, E≈0.85, R≈0.2.[40]
- Pilot: once season-scale telemetry shards and evidencehex show Vt non-increasing and no hard-band breaches under realistic loads, promote lane to PILOT via DID-signed governance shards and confirm K≥0.90, E≥0.90, R≤0.15.[39] [40]
- Production: after successful pilots and independent audit DIDs, promote to PROD lane and freeze schema/corridors for that version, with kerdeployable requiring K≥0.94, E≥0.91, R≤0.13 for deployments.[41] [40]
- Archival (ARCHIVE): old versions move to ARCHIVE lane as immutable history; they remain readable for calibration and audits but cannot be used for new actuation, and any reprocessing must emit new RESEARCH or SIM shards rather than mutating archived data.[41] [40]

Every stage change is a shard event (lane change, new version) with its own evidencehex and ALNSPECHASH, so the lifecycle is fully replayable. K≈0.95, E≈0.91, R≈0.12.[40]

34. `eco_shards` and CSV BOM / non-UTF-8

Your corpus assumes strict RFC4180 CSV and UTF-8 text for qpudatashards; there is no endorsement of non-UTF-8 encodings. Best practice in this stack is for `eco_shards` to treat non-UTF-8 or unexpected BOMs as ingest-time faults: either strip a UTF-8 BOM transparently and proceed, or fail fast with a schema/encoding error that pushes rcalib high and causes `no schema, no write` / `kerdeployable` gates to reject the shard.[39] [41] [40]

Given the emphasis on deterministic hashing for evidencehex and ALNSPECHASH, silently normalizing non-UTF-8 encodings would undermine reproducibility; instead, upstream writers should be required to emit canonical UTF-8 RFC4180, and any deviation should be surfaced as an explicit error shard, not auto-fixed in `eco_shards`. K≈0.88, E≈0.9, R≈0.17.[41] [40]

35. Testing `oss_vfs` under network partitions

For exabyte-scale `oss_vfs`, your docs already recommend avoiding giant real directories and instead testing path and iterator logic against synthetic manifests and mocks. For network partitions where `/mnt/oss` is temporarily unavailable, best practice is to:[39]

- Design `oss_vfs` backends with injectable layers so they can be driven by a mock that simulates EIO/ETIMEDOUT, slow responses, and partial availability, and write property-based tests that ensure all public APIs fail closed, with clear error codes and no panics, when the backend disappears; and
- Add a smaller set of OS-level integration tests that bind `/mnt/oss` to a real remote filesystem (or FUSE) and then deliberately unmount or firewall it, verifying that `oss_vfs` surfaces errors promptly, that higher layers like `eco-ci-validate` degrade to “observe and stop,” and that no partial shards are written during outages.[41] [39]

This mirrors your broader “observe and stop” design for node failures and ensures VFS behavior stays within defined corridors for rthroughput and rfswrite even under partitions. K≈0.92, E≈0.9, R≈0.13.[41]

⁂

36. OI – The placeholder crates are all named `mod_XXX`; what naming convention should coders use when renaming them to meaningful domain names to avoid collisions?

37. DR – Define "corridor calibration evidence"—what specific types of data (lab reports, pilot telemetry, peer-reviewed studies) are acceptable to justify tightening a corridor band?
38. DQ – In `eco-ci-validate`, are corridor tables loaded once at startup or per shard? What is the memory footprint of holding all corridor bands for all regions?
39. RQ – How can we implement a "shard-based lint" for Rust code that checks for patterns known to increase R (e.g., unbounded loops, large allocations) and emits warnings with

KER impact estimates?

40. OI – The `build_scheduler` mentions Lyapunov-gated approval; what happens if a build is approved but a later shard reveals a corridor violation after deployment? Is there a rollback mechanism?

36–40 all slot into existing patterns; below are concise, implementation-level answers.

36. Naming `mod_XXX` placeholder crates

The current pattern keeps placeholders thin and lane-tagged, with semantics centralized in ecosafety core. When renaming `mod_XXX`, use a three-part convention aligned with shard families: `plane_domain_region` or `plane_domain_purpose`, where `plane` is one of `econet`, `cyboquatic`, `bugs_life`, `biomaterials`, `identity`, etc., and `region` is a short, curated token like `phoenix`.[42] [43]

Crate names should be globally unique in the workspace and mirror ALN/spec names, for example `cyboquatic_fog_phoenix`, `biomaterials_tray_phoenix`, `econet_vfs_core`, matching ALN particles like `FOGNodeShard2026` and tray families; CI can then enforce that each crate name is injective with respect to `(plane, domain, region, year)` to prevent collisions across hundreds of crates. K≈0.93, E≈0.9, R≈0.14.[43] [42]

37. Definition: corridor calibration evidence

Corridor calibration evidence is the body of DID-signed data you accept as justification for tightening `safe/gold/hard` bands in `MetricField/CorridorBand` tables. Acceptable evidence types are:[42] [43]

- Lab and bench reports: standardized assays (e.g., LCMS for PFAS, OECD toxicity tests, biodegradation kinetics) captured as qpudatashards or attached via `evidencehex`, with raw measurements and uncertainty mapped into `rcalib` and domain risk coordinates like `rtox`, `rmicro`.[43]
- Pilot telemetry: season-scale shards from real pilot deployments in the same family and region (Phoenix canals, trays, FOG nodes), showing Vt non-increasing and no hard-band breaches under proposed tighter bands.[42] [43]
- External studies and standards: peer-reviewed papers, regulatory limits, and long-term monitoring translated into corridor rows and referenced by `evidencehex` in governance shards (e.g., `PolicyCorridorSpec`, `DecisionLogEntry`).[43] [42]

Tightening must be monotone (bands only move inward), encoded as new versioned corridor tables (e.g., `EcoCoreParametersPhoenix2026vN`) with their own hexstamps and DID approvals; simulations alone stay in RESEARCH lanes until backed by lab or pilot shards. K≈0.95, E≈0.91, R≈0.12.[42] [43]

38. `eco-ci-validate` corridor loading and memory

The design already assumes corridor tables are read-mostly configuration and should be loaded once, not per shard. The recommended pattern is to load one canonical `EcoCoreParameters*.csv` (or a small set, one per region like `EcoCoreParametersPhoenix2026v1.csv`) at startup into an immutable, shared map, e.g., an `Arc<HashMap<varid, CorridorBand>>` plus pre-indexed per-node views for hot loops; this is exactly the "OnceLock + Arc<HashMap>`corridor cache described in your ecosafety core work.[file:2] [44]

Memory footprint stays tiny: even a few thousand `CorridorBand` rows at, say, 100–200 bytes each is well under a megabyte; region-specific variations are either additional rows with a `region` column or additional small tables, but all conform to the same grammar and remain fully resident. This keeps `eco-ci-validate` corridor lookups O(1) and avoids reparsing per shard. K≈0.94, E≈0.9, R≈0.13.[42] [43]

39. Shard-based lint for Rust code with KER impact

You already use shard-based analysis for KER (`kersummary`, `KerWindow`) and have a pattern for emitting shards from tooling. A shard-based lint fits that grammar:[44] [42]

1. Implement a Rust linter crate (e.g., `ecosafety_lint`) that walks HIR/MIR or uses `rustc`/Clippy callbacks to detect patterns that raise R: unbounded or data-dependent loops in hot paths, large heap allocations, blocking IO on critical paths, unchecked `unsafe`, missing backpressure, etc.[43] [42]
2. For each finding, map it to one or more risk coordinates (e.g., `renergy`, `rthroughput`, `rmaterials`, `rsigma`) via a small, versioned `LintToRisk` corridor shard, and compute a local KER delta using your existing `KerWindow` math.[44] [42]
3. Emit a `CodeLintShard2026v1.csv` for each crate or commit, with rows containing `crate`, `path`, `lint_id`, `risk_plane`, `estimated_r_delta`, and a synthesized `KerTriad`; `eco-ci-validate` then ingests these shards alongside physics corridors, so builds with high-R lint profiles can be derated or blocked, and IDEs can surface "this pattern increases R by ~0.02 on renergy".[44] [42]

This keeps the lint non-actuating, fully shard-visible, and allows you to tune severities by editing the Lint→risk corridors rather than the code, consistent with "no corridor, no build" and Lyapunov-gated build approval. K≈0.94, E≈0.9, R≈0.13.[44] [42]

40. Post-deploy corridor violations and rollback

`build_scheduler` already bases approval on a KER summary over a window of steps, enforcing `Vt_{t+1} ≤ Vt_t` and lane-specific K/E/R thresholds before deployment. If later shards (telemetry or audit) reveal a corridor violation after deployment, the ecosafety grammar treats this as a new, negative evidence shard:[44] [42]

- At node level, any shard with `r_j ≥ hard` or rising Vt forces `safestep` into `Derate` or `Stop`, so controllers locally fall back to observe-and-stop behavior under the same invariants

used in Cyboquatic hydrology and tray pilots.[44] [43]

- At governance level, CI replay and sharddiff tools detect the breach by recomputing Vt/KER over the affected window; `kerdeployable` for that configuration goes false, triggering automatic rollback: disabling the offending config or version in deployment shards, and optionally reinstating the last kerdeployable configuration recorded in audit/history shard families.[42] [43]

Because corridor tables and deploy decisions are versioned and hex-stamped, rollback is just switching the active `EcoCoreParameters*` and `NodePlacement` shards back to the last good hexstamps, not mutating history; the new violation shards remain as permanent audit evidence and will keep K lower and R higher until corridors or designs are corrected and re-approved. K≈0.95, E≈0.91, R≈0.12.[43] [42] [44]

⁂

41. DR – Provide a precise definition of "Shard Completeness" values—what are the allowed values (`SIMULATED`, `MEASURED`, `VERIFIED`, etc.), and how do they affect KER and lane eligibility?

42. DQ – In `ecosafety_core::corridor`, the `normalize_coord` function uses a piecewise linear map. What is the exact formula when the metric is "higher-is-better" (e.g., degradation speed)?
43. RQ – Can we design a `shard-fsck` tool that scans `/mnt/oss` for orphaned shards, missing index entries, or hash mismatches, similar to filesystem consistency checks?
44. OI – The `.envrc` file sets `RUST_ROOT`; this variable is not used by standard Rust tooling. Should it be renamed to avoid confusion, or removed entirely in favor of `CARGO_HOME`?
45. DR – Define "node capacity" in `StorageAndComputeNodes2026v1.aln`—what fields specify storage and compute limits, and how are they used in scheduling?

41–45 all have direct hooks into your existing spine; here are precise, implementation-level answers.

41. Shard completeness values and KER / lanes

The docs say “Shard Completeness” is not a free, separate flag but emerges from schema invariants plus lane and `rcalib`. In practice you effectively have three validation levels, expressed via lane and data-quality coordinates:[45]

- SIMULATED: physics and corridors are syntactically complete, but data come from models or long-horizon sims; lane is `SIM` or `RESEARCH`, `rcalib` is high, and `kerdeployable` is always false for any actuating lane even if K/E/R look good.[45]
- MEASURED: shards originate from physical telemetry or lab experiments; they satisfy `NodeShardv1` invariants, have real `evidencehex`, and `rcalib` sits in a moderate band

signalling some uncertainty; these shards can support RESEARCH and PILOT lanes but still cannot make PROD kerdeployable unless corridors and KER pass stricter bands.[46] [45]

- **VERIFIED**: shards are DID-signed, schema-clean, corridor-complete, with `rcalib` in safe/gold bands, and K/E/R meeting lane thresholds; only these are eligible for PILOT/PROD actuation under the lane KER gates (e.g., PROD K≥0.94, E≥0.91, R≤0.13).[46] [45]

In other words, "completeness" is encoded by "all mandatory physics/governance fields present and internally consistent" plus `rcalib` and lane semantics; K grows with higher completeness (more real evidence), E benefits from verified ecoimpact, and R is bounded by refusing to promote high-`rcalib` or SIM shards into actuating lanes. K≈0.96, E≈0.91, R≈0.12.[45] [46]

42. `normalize_coord` for higher-is-better metrics

The corridor grammar already distinguishes direction and uses an affine, piecewise linear kernel `r = (m - s)/(h - s)` anchored at safe and hard bands. For "higher-is-better" metrics like degradation speed, you encode the corridor with `direction = higher_is_better`, and the normalization flips so that better-than-gold maps to low risk:[46] [45]

Let x be the raw metric (e.g., `t90` speed where larger is better), and let `safe_min`, `gold_min`, `hard_min` be corridor edges in that direction. The higher-is-better normalization is:

- If $x \geq \text{safe_min}$, then $r = 0$ (within or above safe band).
- Else if $x \leq \text{hard_min}$, then $r = 1$ (at or beyond hard failure).
- Else (between hard and safe) $r = (\text{safe_min} - x)/(\text{safe_min} - \text{hard_min})$, clamped into $[0, 1]$.[46]

This is just the standard corridor kernel reflected so that higher values move you toward 0 risk instead of 1, and it keeps $r_j \in [0, 1]$ and Lyapunov residuals consistent across "higher-is-worse" and "higher-is-better" metrics. K≈0.95, E≈0.91, R≈0.12.[46]

43. Designing a `shard-fsck` tool

Your stack already sketches a shard lint tool and index strategy over `/mnt/oss` and `/mnt/oss_index`. A `shard-fsck` fits cleanly as a non-actuating Rust CLI that:[45]

- Walks shard trees using an oss-aware walker (not naïve `walkdir`), preferring batched listings and index hints to avoid 10M-file directory thrash.[45]
- For each shard, verifies RFC4180 structure, schema version, ALN spec hash (`ALNSPECHASH`), `hexstamp` consistency, and DID signature, by recomputing the canonical bytes and digest; any mismatch marks the shard as corrupt or tampered.[45]
- Cross-checks shard IDs and paths against the append-only key–value index (`/mnt/oss_index`), flagging "orphaned" data shards (no index entry) and "dangling" index

entries (index points to a missing shard), plus broken lineage where `parentevidencehex` chains don't resolve.[45]

Output is itself a set of governance shards (e.g., `ShardFsckReport2026v1.csv`) with KER-like summaries (`rcalib` high where corruption is found), and CI can then gate deployments on "no critical `shard-fsck` findings in PROD lanes". K≈0.95, E≈0.91, R≈0.12.[45]

44. `RUST_ROOT` vs `CARGO_HOME` in `.envrc`

The current CI and tooling already treat `rustup` and `CARGO_HOME` as canonical; toolchains install under `mnt/oss/cargo`, and workflows never reference `RUST_ROOT`. The docs explicitly note that `RUST_ROOT=/mnt/oss/rust` in `.envrc` is legacy/misleading and should either be removed or repointed, with coders relying on `CARGO_HOME` and standard `rustup` shims instead.[45]

Best move is to delete `RUST_ROOT` from `.envrc` and document `CARGO_HOME` as the only supported control point; if you must keep it for transition, set it to the same directory as `CARGO_HOME` and treat it as deprecated in comments so scripts don't drift onto a separate, unaudited toolchain. K≈0.93, E≈0.88, R≈0.14.[45]

45. Node capacity in `StorageAndComputeNodes2026v1.aln`

The capacity pattern is already shown for quantum nodes: `maxqubits`, `maxcircuitdepth`, `maxsessions` in `QuantumVMRegion`, with corridors translating them into normalized risk coordinates. By analogy, `StorageAndComputeNodes2026v1.aln` encodes capacity as explicit scalar fields on the node shards, such as:[45]

- Storage: `storage_gib`, `iops_max`, optionally `throughput_mbps`.
- Compute: `cpu_cores`, `ram_gib`, `max_concurrent_jobs` or similar.[45]

Schedulers treat these raw capacities as inputs to ecosafety core: corridors define safe/gold/hard bands per metric, normalize into risk coordinates (e.g., `renergy`, `rheat`, `rmaterials` for write amplification), then Lyapunov and KER gates bound how aggressively jobs may be scheduled against a node; overcommitment or sustained high `renergy` / `rheat` pushes R up and can force derating or stop for that node. K≈0.92, E≈0.9, R≈0.13.[45]

⁂

46. DQ – How are `rx_map_json` field updates synchronized across multiple concurrent writers to the same shard family? Is there a locking or merge strategy?

47. RQ – What is the optimal strategy for garbage collecting old `RESEARCH` and `SIM` lane shards that are no longer needed, while preserving auditability?
48. OI – The `eco-ci-validate` binary currently scans all shards; for a coder working on a single crate, this is overkill. How can we scope validation to only the shards relevant to the current change?
49. DR – Provide a definition for "ecosafety invariant orthogonality"—how do we ensure that tightening one corridor does not inadvertently relax another?
50. DQ – In `oss_vfs`, what is the behavior of `list_shards()` when the underlying object store returns an incomplete listing due to eventual consistency?

46–50 all sit on top of patterns you already defined; here are concise answers with KER-style discipline.

46. Concurrent updates to `rx_map_json`

`rx_map_json` is explicitly described as an opaque JSON blob carried in `ResponseShard` / `ecosafety_core` types and parsed into structured maps only at the edges. There is no shared, mutable in-place update to a single shard row: writers are expected to emit new, immutable shards (or new versions) rather than concurrently editing one file, and `oss_vfs::open_shard` is specified to rely on backend atomic "write-then-rename/version" semantics rather than process-level locks.[48] [49]

When multiple tools want to refine or extend `rx_map_json` for the same logical asset, the merge strategy is therefore "write new shard plus lineage," not "lock and patch": each new shard references its predecessor via `parentevidencehex` and possibly lane/evidence fields, and CI uses schema/`rcalib`/KER rules to pick the authoritative one (e.g., latest kerdeployable shard in a lane) rather than trying to merge concurrent JSON fragments. K≈0.9, E≈0.9, R≈0.15. [48]

47. GC for old `RESEARCH` / `SIM` shards

The stack already treats auditability as a hard invariant and reserves explicit capacity (e.g., 10 TB) for long-lived error/audit shards. For `RESEARCH` and `SIM` lanes, the optimal GC strategy is two-tiered:[48]

1. **Cold archiving, not deletion, for anything with governance value.** Shards that were used in decisions, corridor calibration, or publications get migrated into an `ARCHIVE` lane or dedicated archive families (e.g., `ResearchArchiveShard2026v1.csv`), with reduced

replication but preserved `evidencehex`, DIDs, and minimal KER context so replay and lineage remain possible.[48]

2. **Lane-bounded, policy-driven deletion for low-value sims.** Pure exploratory `SIM` shards that never fed into decisions and sit in high `rcalib` / low-K bands can be hard-deleted after a policy window (e.g., 6–12 months), with the policy itself encoded as ALN invariants and logged via audit shards so that the existence and removal of those sims remains visible.[49] [48]

GC jobs operate as non-actuating workloads that read lane, K/E/R, `rcalib`, and references (`parentevidencehex`) to decide which shards can be compacted or deleted, and every removal or compaction emits its own DID-signed audit shard, keeping overall K high and R bounded. K≈0.94, E≈0.9, R≈0.13.[49] [48]

48. Scoping `eco-ci-validate` to relevant shards

You already have two building blocks: a shard-aware watcher producing deltas, and a validation index that tracks last-validated shards. To scope validation for a coder working on a single crate, extend this into a three-step flow:[48]

1. A filesystem watcher or Git diff-based tool emits a `ShardValidationDelta2026v1.csv` listing only shard paths/families touched since the last run (using `inotify` or equivalent, or `git diff` for local changes).[48]
2. `eco-ci-validate` first reads this delta (or a small validation-index shard) and constructs the minimal set of shard families affected by the change, including any that depend on edited ALN specs or corridor tables.[48]
3. The validator then runs full schema/corridor/KER checks only on those families, updating the validation index shards on success; CI lanes still occasionally force a full scan, but the default developer path is "delta only," keeping KER feedback within your 2–5 s interactive corridor.[49] [48]

This preserves "no corridor, no build" for PROD while avoiding exabyte-scale scans on routine dev edits. K≈0.95, E≈0.92, R≈0.12.

49. Ecosafety invariant orthogonality

Your macro and ALN designs already encode a "tightest band wins" rule plus contract versioning, which is exactly orthogonality for invariants. **Ecosafety invariant orthogonality** means:[49]

- Each invariant (corridor row, lane KER gate, Lyapunov rule) is defined over its own coordinates and planes, and tightening one can never implicitly widen or relax another; in conflicts, the strictest applicable constraint is always enforced.[49]
- At compile-time, the `deriveAlnShard` / contract macros parse all relevant ALN versions and enforce that, for any shared coordinate, only the narrowest safegoldhard bands and highest weights are admitted into generated code; mixed major versions are rejected, and

any attempt to bind a type to a looser corridor when a tighter one exists fails compilation.[49]

- At runtime, ALN invariants and CI replay ensure that new corridor shards are monotone-tightening (bands move inward or weights increase, never the reverse) and that tightening in one plane cannot be compensated by hidden loosening elsewhere because Lyapunov weights and KER gates are centrally governed, not per-crate.[50] [49]

This makes “tightening one corridor relaxes another” a structurally non-representable state rather than something policy hopes to avoid. K≈0.96, E≈0.92, R≈0.11.

50. `oss_vfs::list_shards()` under eventual consistency

The corpus notes that `oss_vfs` is designed around logical roots plus streaming iterators and must cope with object-store quirks, but symlink and strong-consistency semantics are explicitly underspecified and flagged as open work. For eventual-consistency listings, the recommended behavior is:[48]

- Treat any single paged listing as advisory, not authoritative; `list_shards()` returns a page plus a KER-like quality coordinate (e.g., `r_namespace` or `r_list_consistency`) and a continuation cursor, with callers aware that some very recent shards might not yet appear.[48]
- Periodically reconcile listings against the append-only index (`/mnt/oss_index`), which is the canonical record of all shards; missing objects in a page but present in the index are treated as transient and retried, while objects listed but absent on fetch are flagged as high `rcalib` / potential corruption for `shard-fsck` to investigate.[48]

Because consistency behavior isn’t fully pinned yet, you should treat this as a design requirement: encode listing consistency as a risk coordinate, require index-backed reconciliation before important operations (e.g., PROD deploys), and document that plain `list_shards()` on eventually consistent stores is *never* used alone as a source of truth. K≈0.6, E≈0.8, R≈0.2 (explicitly acknowledged gap).[48]

⁂

51. RQ – How can we integrate `cargo-audit` and `cargo-deny` into the ecosafety CI pipeline to detect vulnerable or unlicensed dependencies, and should this affect KER scores?

52. **OI** – The placeholder script creates VFS snapshot stubs; are these stubs ever validated, or are they purely documentation? If the latter, they may become outdated and misleading.
53. **DR** – Define the exact format of the "improvement principle trails" stored in the 10 TB audit reserve—what fields are recorded per improvement, and how are they indexed?

54. **DQ** – In `eco_shards::loaders`, how are CSV header rows validated against ALN schemas? What happens if a column is missing or out of order?
55. **RQ** – What is the best approach to implement a "shard replication" mechanism across multiple geographic regions to ensure high availability of `/mnt/oss` for distributed teams?

51–55 tie directly into pieces you already have; here is how they fit the existing grammar.

51. `cargo-audit` / `cargo-deny` in ecosafety CI and KER

Your CI sketches already include a "Security Audit" job that runs a Rust security checker alongside ecosafety checks. Integrating `cargo-audit` and `cargo-deny` cleanly means treating them as first-class CI tools invoked from `toolsecosafety-check.ps1` and `.github/workflows/ecosafety-ci.yml` next to ALN and KER validation, with any non-zero exit failing CI.[51]

In the ecosafety grammar, vulnerable or license-violating dependencies primarily raise governance and operational risk, not physical actuation risk, so they should influence KER as a **separate** coordinate (e.g., `r_supply_chain` or `r_depsec`) that feeds into the Lyapunov residual and R, rather than hard-coding security scores into K or E. A practical pattern is: if `cargo-audit` or `cargo-deny` find unresolved issues in ecosafety-spine crates, raise that coordinate and block kerdeployable for PROD lanes; for peripheral tooling crates, you may allow RESEARCH lane builds but still record elevated R in a KER summary shard so the risk is visible. K≈0.93, E≈0.9, R≈0.13.[52] [51]

52. Validation of VFS snapshot stubs

The existing description of VFS snapshot stubs is that they are ALN-backed manifests of allowed paths and operations, intended for cc-vfs to enforce workspace views. They are not "just documentation": the pattern says unfilled or missing stubs must default to "no corridor, no access," meaning those paths are invisible or blocked rather than silently permitted, and CI should fail if tools try to touch paths not covered by stubs.[52]

What is not yet fully specified is a dedicated validator for stub freshness; your other specs suggest treating each stub as a shard with schema and ALN contracts, then running `alnci` / ecosafety checks over them like any other governed artifact. You can extend this by: (a) adding a `VfsSnapshotShard` ALN schema, (b) validating it in ecosafety CI, and (c) rejecting runs where stubs reference non-existent crates, lanes, or directories, which keeps stubs from drifting into misleading state. K≈0.94, E≈0.9, R≈0.12.[51] [52]

53. Format of "improvement principle trails" in the 10 TB reserve

The 10 TB audit reserve is already defined as long-lived qpudatashard families for CI logs, guard decisions, and governance events with rich KER context. "Improvement principle trails" fit as a specific shard family in that space, for example `ImprovementTrailShard2026v1.csv`, where each row records a single applied improvement and its measured effect over time. Fields the corpus already points to, extended minimally, are:[52]

- Identity: `trail_id`, `asset_id`, `region`, `lane`, `change_kind` (e.g., corridor-tighten, code-refactor, material-swap), `aln_spec_hash`, `evidencehex`, `authordid`, timestamps before/after.[51] [52]
- Baseline vs post metrics: `k_before`, `e_before`, `r_before`, `vt_before`, and `k_after`, `e_after`, `r_after`, `vt_after`, possibly with window descriptors (e.g., 30 d KER window IDs) so you can recompute trends.[52]
- Links: URIs or shard IDs for the underlying DecisionLogEntry, CIOutcome, corridor spec shards, and code commits (e.g., `improvement_source_shard`, `commit_hash`, `corridor_band_id`).[51] [52]

Indexing is via path partitioning by region and time (`…/improvement/region/yyyy/mm/`) plus small index shards per asset listing `trail_id` → shard URI so trails can be replayed quickly without scanning the full 10 TB. K≈0.92, E≈0.91, R≈0.13.[52]

54. `eco_shards::loaders` CSV header vs ALN schema

The shard governance docs state that CSV input must be strict RFC 4180 and validated row-wise against ALN schemas, with "no schema, no write" enforced at ingest. In that pattern, `eco_shards::loaders` is expected to:[53] [52]

- Load the relevant `.aln` schema (e.g., `NodeShardv1.aln`) and construct the canonical header order and types; then check the incoming CSV header line for exact set equality of column names, independent of order.[53] [51]
- If a required column is missing (present in ALN but absent in CSV), ingest fails: a `CorridorIncomplete`/schema error is raised, `rcalib` is driven high, and the shard is rejected from promotion (kerdeployable must be false for PROD).[53] [52]
- If the header columns are out of order but complete, loaders map by name rather than position; order differences are normalized at parse time so internal structs match ALN order, and any unknown extra columns either cause failure or are quarantined, depending on the family's ALN contract.[53] [52]

This matches the "schema as source of truth, CSV as a flat transport" design and keeps unsafe or malformed shards from entering the atlas. K≈0.93, E≈0.91, R≈0.13.[53] [52]

55. Shard replication across regions for `/mnt/oss`

Your own work already points toward treating `/mnt/oss` as a logical namespace backed by multiple physical backends, with routing defined by VFS mapping shards. A shard replication mechanism that fits that design looks like:[52]

- Define a `VfsRoutingShard` or similar family listing each shard family / lane / region and its physical backends (e.g., object-store buckets, volumes), plus replication factor and preferred region; `/mnt/oss` is then a cc-vfs or oss_vfs layer that consults these shards to resolve paths.[51] [52]
- Implement an append-only, non-actuating replication daemon that watches staging or index shards for new objects and pushes them to regional backends, emitting its own

`ReplicationStatusShard` rows with success/failure, lag, and per-region `r_availability` / `r_consistency` coordinates so replication health appears in KER.[52]

- Require that PROD-lane reads consult only replicas marked as fully validated (schema + checksum) and treat partial replication or lag as increased risk, surfaced via KER rather than hidden; for distributed teams, local `/mnt/oss` views are read-through caches over regionally nearest replicas, but authoritative truth remains the hex-stamped shard plus replication index.[51] [52]

This approach keeps shard semantics immutable, lets you add or rebalance regions via shard-level routing changes, and makes availability/consistency another explicit corridor rather than an implicit property of the storage system. K≈0.9, E≈0.9, R≈0.15.[51] [52]

⁂

56. OI – The `agent_interface` vocabulary is limited to three actions; this may force agents to simulate complex workflows through many small steps, increasing shard volume. Is this acceptable at exabyte scale?

57. DR – Provide a definition for "KER target met"—is it a boolean derived from thresholds, or can it have partial states (e.g., "K met, E not met")?
58. DQ – How does `ecosafety_core::residual` handle the case where a coordinate's corridor band is missing entirely? Does it default to a conservative value or fail loudly?
59. RQ – Can we implement a "shard-aware debugger" that allows coders to step through shard validation logic and inspect intermediate rx, Vt, and KER values?
60. OI – The `eco-ci-build.sh` wrapper hardcodes the crate name `eco-ci-validate`; what if a coder wants to validate a different binary or workspace member?

56–60 map cleanly onto things you have partially specified already; here is how they fit.

56. Agent vocabulary and shard volume

The current three-verb vocabulary (`ListShards`, `ProposePatch`, `RunCheck`) is explicitly described as a *safety* constraint: it forces agents to work through shard-centric, reviewable changes instead of arbitrary commands. At exabyte scale, simulating complex workflows as many fine-grained `ProposePatch` calls would indeed explode shard volume and hurt coder/CI ergonomics, even if it remains logically safe.[54] [55]

The recommended evolution is to keep the low-level verbs but add a few higher-order, declarative actions (e.g., `PlanRefactor`, `ApplyRefactorBatch`, `SummarizeKerImpact`) that still produce shards, but batch intent and diffs into structured artifacts checked by CI. This keeps ecosafety invariants and auditability while avoiding a combinatorial explosion of tiny shards;

the main residual risk is in mis-specified plans, mitigated by DID-signed approvals and KER gating before any batch apply. K≈0.93, E≈0.9, R≈0.15.[54]

57. Definition of "KER target met"

The spine already defines lane-specific KER bands (e.g., RESEARCH, PILOT, PROD) and uses `kerdeployable` as a derived gate, not a free flag. In that grammar, "KER target met" is primarily a **boolean** per lane: for a given shard or deployment, you either satisfy that lane's thresholds (e.g., PROD requires $K \geq 0.94, E \geq 0.91, R \leq 0.13$) or you do not, and CI treats that as pass/fail.[55] [54]

However, partial states do exist *for diagnostics*: K, E, and R are tracked separately, and tools like `kersummary` report them individually (e.g., "K met, E not met") so humans and schedulers can see which dimension is blocking. The governing field `kerdeployable` remains a single boolean derived from all three (plus residual and corridor checks); any unmet dimension forces it to `false`, but UIs and shards can still expose more granular "which part failed" metadata. K≈0.96, E≈0.91, R≈0.11.[55] [54]

58. `ecosafety_core::residual` with missing corridor bands

Your docs state "no corridor, no build" and treat missing mandatory corridors as a structural error: a PROD-lane shard with absent mandatory bands is invalid and must be blocked. At the math level, `ecosafety_core::residual` assumes a complete set of weights and normalized r_j; if a coordinate's `CorridorBands` is missing, the earlier answers specify that ingest and CI must fail corridor-presence checks before residual computation, rather than silently fabricating a value.[56] [54]

So the intended behavior is **fail loudly**, not "default conservative": the loader/validator reports a `CorridorIncomplete`/schema error, sets `kerdeployable = false`, and stops that shard from entering any actuation lane. RESEARCH/SIM shards may explore unsafe regions, but even there, missing mandatory bands should show up as elevated `rcalib`/`rsigma`, not as hidden defaults in `residual`. K≈0.95, E≈0.92, R≈0.12.[56] [54]

59. Shard-aware debugger for rx, Vt, KER

You already have most ingredients for a **shard-aware debugger**: `ecosafety_core` exposes pure functions (`normalize`, `compute_residual`, KER windows), and tools like `eco-ci-validate` and `responseshard-emitter` compute KER from shard inputs. A debugger can be layered on top by wrapping these calls in a CLI or IDE extension that, given a shard or crate, re-runs the same validation steps but records each intermediate vector: normalized `rx`, per-coordinate contributions $w_j r_j^2$, `V_t`, and windowed K/E/R.[54] [55]

Practically, this could be a `--debug` mode on `eco-ci-validate` that emits a sidecar `DebugResidualShard2026v1.csv` or JSON for the shard under inspection, plus a VS Code adapter that lets coders step through "normalize → residual → KER" instead of raw code lines. Because all logic is non-actuating and shard-first, this fits your grammar and should

significantly improve explanation and trust without touching controllers. K≈0.94, E≈0.9, R≈0.12.[55] [54]

60. `eco-ci-build.sh` hardcoded `eco-ci-validate` binary

The current shell/PowerShell orchestration scripts call `eco-ci-validate` as the canonical validator, but they already support parameters (e.g., `ToolchainPath`, `Mode`) and are meant to sit in a governed tooling repo. If a coder wants to validate a different binary or workspace member, the clean approach is to generalize the wrapper's interface—add flags like `--validator-bin` and `--package`—and treat those as shard-backed configuration so CI can audit which binaries are considered ecosafety validators.[54] [55]

```
Concretely, `eco-ci-build.sh` can read a small `EcoCiConfigShard` (CSV/JSON) that ma
```

⁂

61. DR – Define "ecosafety corridor update" process—what steps must a coder follow to propose, test, and merge a change to `EcoCoreParameters2026v1.csv`?

62. DQ – In `response_shard_core`, are `knowledge_factor`, `eco_impact`, and `risk_of_harm` stored with a fixed number of decimal places? How does rounding affect `ker_deployable` decisions?
63. RQ – How should we design a "shard migration rollback" procedure in case a migration tool introduces errors that increase R across many shards?
64. OI – The `storage_shards` crate includes `is_safe_for_writes()`; what is the expected behavior when a node is at the edge of its hard band but a critical security patch must be deployed?
65. DR – Provide a definition for "Bostrom DID method" as used in shard signatures—what is the exact DID prefix and resolution method?

61–65 sit right on top of the spine you already wrote; here is the tight, spec-aligned behavior.

61. "Ecosafety corridor update" process

An **ecosafety corridor update** is a versioned change to the canonical corridor tables (e.g., `EcoCoreParametersPhoenix2026v1.csv` / `PolicyCorridorSpec.aln`), never an in-place edit. The coder path is:[57] [58]

1. **Propose** – Fork or branch the shared spec repo (e.g., `.ecosafety-spec`), copy the current EcoCoreParameters file to a new, versioned name (such as `EcoCoreParametersPhoenix2026v2.csv`), and edit only corridor rows (safe/gold/hard,

weights) under ALN governance, keeping schema and varids fixed. The change must be anchored with new `evidencehex` rows referencing lab/pilot shards and must update the ALN particle (e.g., `PolicyCorridorSpec`) with the new corridor set ID and epoch.[58] [57]

2. **Test in RESEARCH/PILOT** – Point RESEARCH or PILOT lanes at the new corridor set by updating KER context / corridor IDs (e.g., `KERCONTEXTPhoenixEcoSafetySpine2026v1` → …`v2`) and run `eco-ci-validate` plus replay/atlas jobs against representative shards. CI must show: non-increasing V_t outside the safe interior, no new hard-band breaches, and K/E/R staying within lane thresholds for the test window.[59] [57]
3. **Governance review** – Package the proposal as a shard (e.g., `CorridorChangeProposal2026vN.csv`) listing corridors changed, expected K/E/R deltas, affected assets, and evidence shards; get it DID-signed by the relevant governance DID (e.g., `did:bostrom:artemis-production-lane`) according to `EcoSafetyKernel` and governance specs.[57] [58]
4. **Merge and switch lanes** – Once K≥0.9, E≥0.9, R≤ lane limit over the replay window and reviewers sign off, merge the spec branch and update CI configuration so PROD controllers and `eco-ci-validate` read the new EcoCoreParameters version. Rollback is defined as switching CI back to the previous corridor file, since old versions remain immutable; no mutation occurs in place.[58] [57]

K≈0.95, E≈0.91, R≈0.12 for this process.

62. Decimal places and rounding in `response_shard_core`

In `response_shard_core` and related ResponseShard schemas, `knowledge_factor`, `eco_impact`, and `risk_of_harm` are modeled as floats in $[0, 1]$ (typically `f64`), not fixed-precision strings; CSV/JSON are just encodings. The math that drives `ker_deployable` uses the underlying floating-point values and explicit thresholds (e.g., PROD needs $K \geq 0.94, E \geq 0.91, R \leq 0.13$); CI compares against these thresholds with a small epsilon, not by string formatting.[57] [58]

Rounding only enters at the presentation layer (e.g., printing KER with two decimals in logs), and should not flip a deploy decision, because the gate uses the raw values plus an explicit tolerance band documented in `EcoSafetyKernel` and `safestep` configs. For highly sensitive promotions, you already plan to tighten determinism (ordered sums, optional fixed-point) so that borderline cases are reproducible across platforms, but the *spec* assumes rounding is non-governing. K≈0.9, E≈0.9, R≈0.16.[57]

63. Shard migration rollback when R spikes

Your own guidance on corridor/parameter rollback is the right pattern for **shard migration rollback** as well. Design it as:[57]

1. **Versioned migrations only** – A migration tool must always write **new** shard versions (new files or new versioned families) with their own `evidencehex` and possibly new `KERCONTEXT`, never overwrite or delete old shards. This guarantees you can compute before/after KER deltas and revert by switching readers back to the old family.[59] [57]

2. **Post-migration KER diff** – Immediately after a migration run, a `sharddiff` tool recomputes r_x, V_t, and K/E/R for affected families under the same corridors and emits summary shards (per family/topic) with ΔK, ΔE, ΔR; CI fails if any PROD-lane asset has K or E below threshold or R above its band.[59] [57]
3. **Automated rollback switch** – If CI flags increased R beyond allowed bounds, rollback is implemented by:
   - pointing atlas/index readers, controllers, and CI back to the previous shard family or version ID (via VFS routing or schema config), and
   - marking the migrated family's lane as RESEARCH or quarantined so it can't feed actuation.[59] [57]

   Because original shards are immutable, rollback is just configuration + routing change, not data surgery.[57]
4. **Forensics and governance** – Migration runs and rollbacks are themselves captured as qpudatashards (e.g., `ShardMigrationRun2026v1.csv`, `ShardMigrationRollback2026v1.csv`) with KER fields so you can analyze why R increased and prevent recurrence.[57]

K≈0.95, E≈0.9, R≈0.12.

64. `is_safe_for_writes()` at hard-band edge with critical security patch

`storage_shards::is_safe_for_writes()` is specified to align with corridor and KER gates: writes are safe only when key storage-risk coordinates (e.g., `r_energy`, `r_materials`, `r_heat`, `r_rf`) are comfortably below hard bands—effectively in safe/gold regions with margins defined in the corridor spec. At the **edge** of a hard band, the function is expected to return false and block writes, treating any further risk as unacceptable.[57]

For a critical security patch, the framework's answer is **governance, not a hidden override**: you apply the "research/pilot lane override" pattern—route the patch through a non-actuating or degraded-mode lane with explicit KER justification—or temporarily derate other workloads to bring storage coordinates back into a safe band before applying the patch. Embedding a special-case bypass into `is_safe_for_writes()` would violate "no corridor, no build"; instead, any emergency override must be a DID-signed, lane-encoded decision shard that's auditable and still respects Lyapunov and hard-band semantics. K≈0.94, E≈0.9, R≈0.13.[59] [57]

65. "Bostrom DID method" definition

The **Bostrom DID method** in your stack is the canonical identity scheme for all shard-level signatures and crate authorship. In practice:[58] [59]

- **Prefix / method name** – DIDs are of the form `did:bostrom:<method-specific-id>`, with examples like `didbostrommt6883-cybernetic` in docs and `shard_did_signature_mandatory` invariants requiring that `didsignature` fields "start with `didbostrom`".[58] [57]
- **Use in shards** – Shard schemas (e.g., `NodeShardv1`, `DecisionLogEntry`, `EcoEvidenceCrate`) require a DID field; CI and schema validators enforce the prefix rule and tie DIDs to

hex-stamped specs and crates, so every ecosafety decision can be traced to a Bostrom identity.[58] [59] [57]

- **Resolution** – The precise network resolution mechanics (on-chain lookup, HTTP resolver, etc.) are not fully specified in the provided text; what *is* specified is that the DID, its associated `evidencehex`, and any on-chain record form a cryptographically verifiable trail from code to shards to governance decisions. Implementations are expected to use a Bostrom ledger to resolve `did:bostrom:*` to public keys and metadata, but the method spec itself lives outside the excerpts you have here.[59] [58]

K≈0.93, E≈0.91, R≈0.13; remaining risk is in the external DID method spec and resolver, which your corpus treats as a given external trust anchor.

⁂

66. DQ – In `eco_shards::response_turn`, is `Vt_before` required to exactly match the `Vt_after` of the previous shard in the sequence? What tolerance is allowed?

67. **RQ** – Can we implement a "shard-based code coverage" tool that correlates test coverage with KER improvements, showing which code paths most affect ecosafety?
68. **OI** – The placeholder script generates ALN files with simple key-value format; if a coder forgets to upgrade them to full HALN, will CI catch this, or could invalid specs slip through?
69. **DR** – Define "evidence hex stamp" algorithm more concretely—if it includes spec content and protocol ID, what is the exact serialization and hash function?
70. **DQ** – How does `oss_vfs` handle file locking? Is there any advisory or mandatory locking to prevent concurrent writes to the same shard file?

Answering 66–70 in order, aligned with the spine you already have.

66. `Vt_before` vs previous `Vt_after` in `eco_shards::response_turn`

`ResponseShardEcoTurn` is documented to treat V_t as a continuous Lyapunov sequence across turns, but it does not require exact bitwise equality between `Vt_before` and the previous shard's `Vt_after`. Instead, the invariant is that `Vt_before` for a turn must be derived from the canonical previous state (atlas window or last accepted turn) within a small epsilon, because the ecosystem already uses tolerances to handle floating-point non-associativity.[60]

Practically, you should enforce that:

- `Vt_before` ≈ previous `Vt_after` within a configured tolerance `eps_vt_link`, and
- safestep still enforces $V_{t+1} \leq V_t$ outside the safe interior and hard bands.[61] [60]

A defensible choice is to treat any discrepancy larger than the same `eps_vt` used in `SafeStepConfig` as a schema or pipeline error: reject the shard or recompute `Vt_before` from

the canonical RiskVector rather than trusting a mismatched value. This keeps the Lyapunov proofs intact without demanding impossible bit-exact continuity across platforms.[60] [61]

67. Shard-based code coverage correlated with KER

You can implement a **shard-based code coverage** tool as a non-actuating Rust crate that joins three streams that already exist in your stack: per-test coverage, per-change KER summaries, and shard families.[61] [60]

The design is:

- During CI, run tests with coverage instrumentation and emit a coverage shard per crate or file, using the same qpudatashard pattern (CSV with hexstamp, didsignature, lane, etc.). Each row includes `file_path`, `function`, `lines_covered`, and a stable code-element ID.[61]
- For each change (PR or commit), `eco-ci-validate` or a sibling tool already computes K/E/R deltas over affected shard families using the shared ecosafety math.[60] [61]
- A new "coverage-KER" tool then correlates the two: for each code element, it looks at which tests executed it and how K/E/R changed in those tests' KER windows, emitting a summary shard like `CoverageKerImpact2026v1.csv` with fields such as `code_id`, `ker_delta_k`, `ker_delta_e`, `ker_delta_r`, `n_tests`.[60]

Because shards are the indexing primitive, you can then query "which functions are associated with the largest R reductions or E gains in biodegradable or Cyboquatic corridors" by reading this shard, not the raw coverage files. The tool remains non-actuating and fits the "just by researching it" pattern by steering coder attention towards code that empirically improves ecosafety.[61] [60]

68. Placeholder ALN vs full HALN in CI

Your current pipeline clearly separates ALN schemas (canonical grammar) from CSV/JSON shard emitters, and it already uses an `alnci`-style validator as a mandatory tool in the ecosafety kernel's implementation requirements. The placeholder script that emits simple key-value ALN-looking files is therefore only CI-neutral if those outputs are never treated as real specs; once a file is meant to be a schema or contract, CI should absolutely catch an under-specified "toy ALN" form.[61]

Concretely:

- ALN contracts and schemas (e.g., `MetricField.aln`, `PolicyCorridorSpec.aln`, `EcoSafetyKernelv1.aln`) must pass `alnci` validation against the project's `ALN-contract-v1` and schema grammars before any crate depending on them can compile.[61]
- The docs explicitly treat ALN as the single source of truth and note that shard emitters are validated against these specs in build scripts or CI.[61]

So if a coder forgets to upgrade a placeholder, that file either will not be referenced by any crate (harmless but unused) or, if it is wired into the build, `alnci` should fail with schema

violations and block the pipeline. To avoid accidental slip-through, you should enforce a rule like "no file under `alncontracts/` or `alnschema/` may use the placeholder key-value format" and have CI reject any ALN file that lacks required sections, version fields, or evidence hex stamps. That keeps invalid specs from ever entering KER windows.[61]

69. "Evidence hex stamp" algorithm

The **evidence hex stamp** is already partially specified in your contracts and tooling: shards and contracts carry a `hexstamp` field defined as a content hash used in DID signing, and your ResearchPlan emitter illustrates a concrete implementation.[61]

The effective algorithm is:

1. **Canonical serialization**
   Serialize the object's content excluding signature-like fields (e.g., `hexstamp` itself and `didsignature`) using a deterministic procedure: for JSON, pretty-printing with a fixed key order; for CSV, RFC 4180 rows with a fixed header order and newline convention; for ALN, the normalized textual form produced by your ALN tooling.[60] [61]
2. **Hash function**
   Compute the SHA-256 digest of that serialized byte stream and encode it as a lowercase hexadecimal string; this is exactly what `sha256digest` does in the ResearchPlan emitter, where `hexstamp` is set to `sha256(input_raw)`.[61]
3. **Embedding**
   Store this hex string in the shard's `hexstamp` field and in contract `evidencehex` anchors, and require CI to recompute the hash and compare it to the stored value (`shardhexstampimmutable` invariant).[61]
4. **Binding to DIDs**
   Bostrom DID contracts then sign or otherwise bind this `hexstamp` into a DID document or signature record; ALN invariants already require that shards be signed by a DID starting with the correct method prefix and that `hexstamp` match the content hash at validation.[61]

So the "evidence hex stamp" as used in your system is "SHA-256 over canonicalized content, stored as a 64-character hex string in `hexstamp` / `evidencehex`, validated by CI, and referenced from Bostrom DID signatures." If you want to tighten this further, the serialization rules should be written explicitly into an ALN contract so shard producers and validators cannot diverge.[61]

70. File locking in `oss_vfs`

The VFS and storage docs describe `oss_vfs` as a logical, shard-aware view over an underlying POSIX-like filesystem or object store, emphasizing path length constraints, directory structure, and efficient traversal, but they do not specify an elaborate locking protocol. Shards themselves are treated as immutable once written, and most mutating operations are staged into separate areas (e.g., `mnt/oss/staging`) before promotion, which reduces the need for in-place locking.[60] [61]

Given those constraints, `oss_vfs` should behave as follows:

- For normal shard trees under `mnt/oss`, writes are append-only or copy-on-write, and concurrent modifications to the same shard file are treated as a violation of the shard lifecycle rather than arbitrated by VFS locks; CI and promotion tools enforce "no in-place edits" and DID hexstamp invariants instead of relying on advisory locks.[61]
- Any necessary coordination (e.g., preventing two migration tools from writing the same target path) should be handled at a higher level via atomic renames or object store semantics, not by a custom mandatory locking layer in `oss_vfs`.[60]

In other words, there is no separate advisory or mandatory locking API exposed for shard writers; the safety model is "immutable shards plus staged writes and CI gating," not "shared mutable files with locks." Where you do need mutual exclusion (for index shards, for example), you should use atomic operations provided by the backend (e.g., write-once keys, prefix-based partitions) and keep those rules explicit in the indexing service rather than buried inside `oss_vfs` itself.[60] [61]

KER scoring for this answer

- **Knowledge factor**: K ≈ 0.95 — directly grounded in your ecosafety spine, ALN contracts, and tooling descriptions.[60] [61]
- **Eco-impact**: E ≈ 0.91 — clarifying continuity of V_t, evidence stamping, and CI validation improves reliability of ecosafety gates "just by researching it." [62] [60] [61]
- **Risk of harm**: R ≈ 0.12 — residual risk lies in floating-point tolerances, hash canonicalization, and concurrent tooling, all surfaced and controlled via existing invariants and shard lifecycles.[60] [61]

⁂

71. RQ – What is the optimal shard file size distribution for exabyte-scale storage to balance listing latency and throughput? Should very small shards be coalesced?

72. **OI** – The `eco-ci-validate` binary uses `serde_yaml`; YAML is known for security issues with deserialization. Is this dependency safe in a CI context, or should it be replaced with a simpler config format?
73. **DR** – Provide a definition for "corridor mandatory flag"—what criteria make a corridor mandatory versus optional, and how is this decided?
74. **DQ** – In `ecosafety_core::invariants::safestep`, is the decision `Derate` or `Stop` based solely on `Vt` increase, or does it also consider the magnitude of increase?
75. **RQ** – How can we implement a "shard heatmap" visualization for coders that shows which topics or regions have the highest R and lowest K/E, guiding research priorities?

Answering 71–75 in order.

71. Optimal shard file size distribution

Your docs deliberately avoid a single "magic" shard size and instead frame exabyte-scale behavior in terms of directory cardinality, streaming iterators, and IO corridors rather than fixed MB values. The guidance is: keep **per-directory entry counts in the tens of thousands, not millions**, and treat overly fine sharding (too many tiny files) as a risk coordinate (`r_namespace`) to be kept within band.[63]

Given that shards are RFC 4180 CSV streams and compute usually dominates parsing, a good pattern is to keep **logical shards sized for natural windows** (e.g., per day, per basin, per node, per topic) so each file has many rows but not so many that listing or scanning becomes pathological. Very small shards (dozens of rows or kilobytes) proliferate directory entries and hurt listing latency; your own research notes that namespace size and directory listing throughput must be treated as corridor variables and that exabyte-scale readiness depends on bounding these. So yes: in PROD lanes you should coalesce very small shards into larger, time- or topic-batched shards until directory sizes stay within measured safe corridors, and treat r_namespace (too many tiny files) as a first-class input to KER for storage governance.[64] [63]

72. `serde_yaml` in `eco-ci-validate`

The current stack uses `serde_yaml` exclusively for **configuration** (CI profiles, schema lists, lane thresholds), not for shard payloads or user-supplied arbitrary documents. Shards remain CSV/JSON and go through their own validators; YAML is a small, human-authored control surface.[65] [63]

In that role, the dependency is relatively safe, provided you:

- Treat YAML inputs as **trusted repo config**, not as arbitrary, network-supplied data.
- Avoid advanced YAML features (anchors, custom tags, implicit code execution); `serde_yaml` by default does not execute arbitrary code, it just deserializes into Rust types, which aligns with your use.[63]

If you want to reduce attack surface further, you could later migrate configs to a more restrictive format (e.g., TOML or JSON), but there is no indication that YAML here is a current KER driver of harm. The bigger risk is config drift, which you already mitigate via DID-anchored specs and CI validation of schemas rather than the YAML format itself.[65] [63]

73. Definition of "corridor mandatory flag"

The **mandatory flag** is already modeled on your `MetricField` and `CorridorBand` grammar: each metric row includes `mandatory: bool`, and schemas plus CI treat this as "must be present and honored in any actuation lane."[63] [65]

Criteria that make a corridor mandatory:

- The metric participates directly in Lyapunov residual $V_t = \sum_j w_j r_j^2$ and KER gates for ecosafety (e.g., toxicity `rtox`, microresidue `rmicro`, PFAS residue, key hydraulics, neurorights-relevant fields).[64] [65] [63]
- Governance has decided that absence or silence on this dimension is unacceptable in PROD/PILOT lanes; ALN invariants like `corridorpresent` and `corridormandatoryallactuation` enforce that every actuation contract references all mandatory corridors.[65]

How it is decided:

- Corridor specs (`PolicyCorridorSpec.aln`, `MetricField.aln`) carry `mandatory=true/false`, and ALN contracts plus CI tests reject any shard in a PROD lane that lacks a row for every `mandatory=true` metric (`no corridor, no build`).[63] [65]
- Optional corridors are those used for diagnostics, research, or secondary objectives (e.g., some performance metrics) where missing data increases `rcalib` but does not make actuation structurally invalid; these have `mandatory=false` and appear in RESEARCH/SIM lanes first.[64] [63]

So, "mandatory corridor" means: **required for any actuation contract in the relevant plane and lane; its presence and valid ordering are enforced by schema and CI, and its absence structurally blocks kerdeployable for PROD.**[65] [63]

74. `safestep`: Derate vs Stop

Your `safestep` logic has **two layers** of decision, only one of which uses V_t.[64] [63]

1. **Hard coordinate bands**: if any normalized coordinate has $r_j \geq 1.0$ (beyond `hardmax` within a small tolerance), `CorridorDecision` is `Stop` immediately, independent of how large the increase is in V_t. This is the "hard band immediate stop" invariant and is also echoed in ALN contracts (`corridornohardmaxbreach`, `emergencystopimmediateonhardmax`).[63] [64] [65]
2. **Residual change with tolerance**: outside a small safe interior band on V_t, the rule is "non-increase" up to a tiny tolerance; if `next_vt` exceeds `prev_vt` beyond that tolerance, the decision is `Derate`, not `Stop`. The magnitude of the increase matters only insofar as it exceeds the configured epsilon; there is no second threshold at which a large but sub-hardband increase flips from `Derate` to `Stop` purely based on V_t.[64] [63]

So:

- **Stop** is driven by hardband breaches (any r_j at/over hardmax) or explicit emergency invariants, not by the size of a V_t increase.[65] [63]
- **Derate** is used when V_t increases beyond epsilon while all coordinates stay within bands; the system backs off actuation level but does not hard-stop solely on magnitude.[63] [64]

75. Implementing a “shard heatmap” for K/E/R

You already have all the ingredients: qpudatashards that carry topic/region, K/E/R, lane, and evidence, plus KER aggregate shards for fast queries. A “shard heatmap” can be implemented as a non-actuating visualization pipeline:[64] [65] [63]

1. **Aggregation layer**
   Extend the existing KER aggregate shard family (`KerAggregateShard2026v1.csv`) so each row includes fields like `topic_id`, `region`, `lane`, and aggregated `k_avg`, `e_avg`, `r_avg`, `n`, possibly with quantiles. CI already maintains such aggregates incrementally when new evidence or decision shards land.[63]
2. **Index and query**
   Use your shard-backed index service to answer queries like “all PROD shards in Phoenix in last 90 days with `R` above 0.15” or “topics with lowest `K` and `E` in RESEARCH lane,” by reading only these small aggregate shards instead of scanning raw data.[65] [63]
3. **Visualization service**
   A Rust (or Rust-fronted) CLI/HTTP service reads the aggregate shard, maps `region` or `topic` onto a 2D layout (map tiles, grid by topic vs time), and emits static PNG/SVG heatmaps or JSON that IDE extensions can render. Color encodes `R` (hot where R is high), saturation or size encodes `K/E`, with filters for lane and corridor family; because outputs are non-actuating and derived from shard data, they fit your governance rules.[64] [63]

This directly supports “just by researching it”: coders can see where `R` is highest or `K/E` are weakest and prioritize new research plans and corridor tightening where it matters most, while all underlying numbers remain KER-gated and DID-anchored.[65] [63] [64]

KER for this answer

- K ≈ 0.95 — definitions and designs are directly grounded in your ecosafety spine, KER math, and tooling docs.[63] [64] [65]
- E ≈ 0.91 — clarifying shard sizing, mandatory corridors, safestep semantics, and KER-driven heatmaps improves ecosafety governance and coder focus.
- R ≈ 0.12 — residual risk lies in corridor calibration (e.g., r_namespace bounds, Vt tolerances) and visualization interpretation, all surfaced and bounded by existing KER and lane rules.[64] [65] [63]

⁂

76. OI – The `agent_interface` restricts writes to staging; how does the promotion process handle shards that reference other shards (e.g., via `corridorid`)? Must all referenced shards also be promoted together?

77. DR – Define "Lyapunov weight" selection process—how are initial weights chosen for new risk coordinates, and how are they recalibrated over time?
78. DQ – In `eco_shards::biodegradable_substrate`, how are `mass_loss_28d` and `mass_loss_180d` normalized into `rbiodegspeed`? What is the mapping formula?
79. RQ – Can we design a "shard-based time travel" query interface that allows coders to see the KER landscape at any past commit, for debugging regressions?
80. OI – The `setup_mnt_oss.sh` script uses `sudo`; in containerized CI environments where `sudo` is unavailable, how should the VFS be set up?

Answering 76–80 in order.

76. Promotion from staging when shards reference other shards

The documented pattern is: **agents can only append to `mntoss/staging`, and a separate promotion tool/daemon validates and copies approved shards into the live qpudatashard tree.** Shards already carry linkage fields such as `corridorid`, `ALNSPECHASH`, `lane`, and `evidencehex`, and promotion is governed by ALN contracts plus `no schema, no write` and KER gates.[66] [67]

For shards that reference other shards (via `corridorid`, validation index IDs, etc.), promotion must ensure **referential integrity into the PROD namespace**:

- If the reference is to a **parameter/corridor shard** (e.g., `EcoCoreParametersPhoenix2026vN`, corridor tables in `PolicyCorridorSpec`): those parameter shards must already exist in the authoritative spec repo and/or PROD corridor family, so the new shard can simply reference the existing versioned ID.[67] [66]
- If the reference is to **other new data shards** still in staging, the promotion job must promote a **closed set** together: either all mutually referenced shards are promoted in one transaction, or the promotion fails.[66]

Practically, your promotion tool should:

1. Read staged shards, collect all `corridorid`/shard IDs they reference.
2. Check that each target either already exists in PROD or appears in the same staged batch.
3. Only move shards whose entire reference graph resolves; anything with dangling references stays in staging or is quarantined with high `rcalib`.[67] [66]

So not “always promote everything together”, but **never promote a shard whose references would point to non-existent or staging-only objects.**[66] [67]

77. Lyapunov weight selection and recalibration

Your spine fixes the residual as $V_t = \sum_j w_j r_j^2$ over normalized $r_j \in [0, 1]$. Weights w_j are not arbitrary: they are part of the **MetricField / corridor catalog** and are governed like any other parameter.[68] [67]

Initial selection for a new risk coordinate:

- A new `MetricField` row is created with units, bands, direction, and an initial `weightw`, based on **dimensional importance and normative policy**: toxicity, irreversible damage, and neurorights fields typically receive higher weights than performance or nuisance metrics.[68] [67]
- The sum and scale of weights are chosen so that typical safe/gold operation yields V_t comfortably below 1 and hard-band breaches dominate the residual.[68]

Recalibration over time:

- Weights live in shard-governed corridor tables (e.g., `MetricField`, `CorridorBand` shards) and are versioned; you never mutate in place but emit new versions with new `evidencehex` and `epochgrammarversion`.[67] [68]
- Replay jobs and calibration studies (Phoenix pilots, biodegradation labs, quantum pilots) compute how changes in each coordinate affect ecological outcomes; if a variable turns out to be under- or over-emphasized, its weight is adjusted in a **new corridor version**, then replayed on historical shards to ensure K, E, R movements are monotone in the intended direction.[67] [68]
- Only when replay KER remains acceptable (e.g., no unsafe design becomes more deployable) is the new weight set admitted to PILOT/PROD corridors.[68] [67]

So the **process** is: propose weight in `MetricField`, back it with evidence, test via shard replays, hash and DID-sign the new corridor spec, then roll it out by updating `corridorid`/KER context—not by tweaking constants in code.[67] [68]

78. Mapping `mass_loss_28d` / `mass_loss_180d` to `rbiodegspeed`

The substrate channel docs treat degradation speed as a normalized risk coordinate similar to `rt90`, based on mass-loss over standardized time windows (e.g., 28 and 180 days). The exact variable names differ by schema, but the pattern is:[66] [68]

1. Compute an effective degradation fraction (or time-to-threshold) from mass-loss metrics, e.g.
   - $f_{28} = \frac{\text{mass_loss_28d}}{\text{initial_mass}}$
   - $f_{180} = \frac{\text{mass_loss_180d}}{\text{initial_mass}}$.[66] [68]
     These are constrained to $[0, 1]$ and physically impossible values raise `rcalib` and/or map directly to high risk.[68] [66]
2. Project those observables onto a **corridor in days** (or % loss) with safegoldhard bands, then use the standard piecewise-linear **CorridorBands.normalize** kernel: for a variable x

(e.g., an implied t_{90} or required days to reach a target mass loss),

  - $r = 0$ at or better than `safemax` (fast enough, biodegradable),
  - $r = 1$ at or worse than `hardmax` (too slow, effectively persistent),
  - linear interpolation between.[67]

For `mass_loss_28d` / `mass_loss_180d` the mapping is aligned with corridor semantics:

- High mass loss within 28/180 days → **low** `rbiodegspeed` (good, near 0).
- Low mass loss over 180 days → **high** `rbiodegspeed` (bad, near 1).[66] [68]

The **formula** is the same as in `kercore::CorridorBands::normalize`: piecewise linear between safegoldhard thresholds in the chosen degradation metric, with clamping at 0 and 1; the project's text does not expose a different bespoke formula for `rbiodegspeed`, it reuses the corridor normalization kernel. Exact band values are read from the biodegradable substrate corridor tables, not hardcoded in code.[68] [66] [67]

79. "Shard-based time travel" KER interface

You already treat KER, corridors, and specs as **versioned shard artifacts tied to commits and epochs**, and ResponseShards record K, E, R plus commit hash and lane. That is exactly the substrate needed for "time travel":[66] [67]

Design sketch:

- At each CI run, emit a **KER summary shard** `KerSummaryRepoCommit2026v1.csv` with fields: repo, commit hash, lane, K, E, R, Vt, KER context, plus `evidencehex` and `ALNSPECHASH`.[67] [66]
- Maintain an index shard family mapping commits to KER summaries and corridor versions; this is small compared to raw telemetry.[66]

A "time-travel" query interface (CLI or IDE integration) can then:

- Given a Git commit, locate the matching KER summary shard, corridor version (`EcoCoreParameters…`), and relevant qpudatashards.
- Render the **KER landscape at that commit**: per topic/plane K/E/R, which corridors were active, which shards were kerdeployable.[67] [66]
- Optionally diff two commits by comparing their KER summary shards and corridor specs, showing which metrics' bands or weights changed.

All of this is non-actuating: it only reads archived shards and specs and is fully auditable because every summary is hex-stamped and bound to a specific commit. So yes, a shard-based time-travel KER interface is not only possible but consistent with your existing ResponseShard/KER summary design; the main work is to standardize the summary schema and build a thin query/visualization layer over it.[66] [67]

80. `setup_mnt_oss.sh` and `sudo`-less container CI

Your docs already note that the current `setup_mnt_oss.sh` pattern (bind-mounting a repo into `/mnt/oss` with `mount --bind` and `sudo`) is **not viable** for exabyte deployments and needs a more abstract VFS routing layer. In containerized CI, you usually do not have `sudo` or `mount` anyway.[66]

The recommended pattern is:

- Treat `mntoss` as a **logical root inside the container's existing filesystem**, not a privileged OS mount. For CI containers, you can simply create `/mnt/oss` as a regular directory owned by the CI user and **check out or sync the governed tree there**.[67] [66]
- Use your user-space VFS implementation (e.g., cc-vfs or an OSS client) inside the container to map logical shard families to physical storage (local volumes or remote object stores), driven by VFS map shards, without requiring kernel-level mounts.[66]
- Adjust scripts so that in CI they skip `sudo` and `mount` entirely: `setup_mnt_oss.sh` (or a CI-specific variant) should detect "container mode" and just ensure the expected directory layout and configuration shards exist, letting tools treat `/mnt/oss/...` as ordinary paths.[67] [66]

In other words, **in containers, VFS setup becomes a matter of directory creation and user-space routing, not privileged mounts**, which keeps CI portable and respects your no-hidden-side-effects governance posture.[67] [66]

KER for this answer

- K ≈ 0.95 — grounded in your existing shard lifecycle, corridor/MetricField grammar, biodegradable substrate semantics, and KER summary tooling.[68] [66] [67]
- E ≈ 0.91 — clarifying promotion rules, weight calibration, biodegradation mapping, time-travel KER, and sudo-less VFS setup directly supports safer, more efficient ecosafety workflows.
- R ≈ 0.12 — residual risk resides in precise corridor calibration for biodegradation, reference-closure bugs in promotion, and CI/container configuration, all explicitly surfaced and bounded by existing invariants and KER gates.[68] [66] [67]

⁂

81. DR – Provide a definition for "ecosafety lane promotion"—what is the exact procedure and required approvals to move a shard family from RESEARCH to PILOT, and from PILOT to PROD?

82. **DQ** – In `eco_shards::response_turn`, how are `corridor_update_ids` and `equation_update_ids` encoded? Are they comma-separated lists or JSON arrays?
83. **RQ** – How can we implement a "shard deduplication" service that identifies identical shards (same content hash) and replaces them with hard links or references to save storage?
84. **OI** – The placeholder crates include `#![forbid(unsafe_code)]`, but the coders may need `unsafe` for FFI to interact with hardware. How should such crates be marked and governed?
85. **DR** – Define "ecosafety audit trail" retention policy—for how long must decision logs and corridor change shards be kept, and in what format?

Answering 81–85 in order.

81. Definition of "ecosafety lane promotion"

Ecosafety **lane promotion** is the governed process of moving a shard family (or asset placement) between lanes RESEARCH → PILOT → PROD, under fixed KER bands and evidence requirements. Lanes and thresholds are:[69] [70]

- RESEARCH: roughly $K \geq 0.85, E \geq 0.85, R \leq 0.20$, non-actuating; used for experiments, calibration, and SIM traces.[69]
- PILOT: $K \geq 0.90, E \geq 0.90, R \leq 0.15$, limited, tightly monitored actuation.[70] [69]
- PROD: $K \geq 0.94, E \geq 0.91, R \leq 0.13$, full deployment with strict CI and external audit.[70] [69]

Procedure RESEARCH → PILOT (per shard family):[69] [70]

1. **Season-scale evidence:** run real or lab pilots (or high-fidelity sims for non-actuating lines) emitting qpudatashards in the same family (e.g., NodeShard, BiodegradableSubstrateShard, ResponseShardEcoTurn) showing that under realistic loads:
   - No mandatory coordinate breaches its hard band.
   - Outside the small safe interior, V_t is non-increasing over the evaluation window.[70] [69]
2. **KER recomputation:** recompute K, E, R over a defined window (e.g., months of operation) using the canonical ecosafety core, with the appropriate corridor set and KER context (e.g., `PhoenixEcoSafetySpine2026v1`).[69] [70]
3. **Governance & signing:** ensure the family meets PILOT thresholds and then:

- Update lane from RESEARCH to PILOT in the governing ALN spec / NodePlacement shard.[69]
- Have at least the owning DID sign the corridor set and placement shard; for actuation-bearing families, require an independent auditor DID as well.[70] [69]

Only then can controllers treat shards in that family as PILOT; type-level APIs and CI enforce that only `LanePilot` placements are admissible to pilot kernels.[69]

Procedure PILOT → PROD: identical structure, but with **stronger evidence** and **stricter bands**: [70] [69]

1. **Longer and broader telemetry:** multi-season or multi-year operation across varied conditions, showing stable or declining residual V_t and absence of hard-band violations. [69]
2. **KER window meets PROD band** $K \geq 0.94, E \geq 0.91, R \leq 0.13$ over the specified window, including explicit uncertainty (`rsigma`, `rcalib`) planes.[70] [69]
3. **Governance approvals:** promotion to PROD lane requires:
   - Updated ALN corridor specs and NodePlacement shards with lane=PROD.
   - DID signatures from the operator plus at least one independent auditor, and often a quorumsigned governance shard documenting the decision.[70] [69]

No lane promotion is done by flipping a flag in code; it is always via new, hex-stamped shards and ALN specs, with CI and type-level constraints preventing use of non-PROMOTED assets in PROD deploy decisions.[71] [69]

82. Encoding of `corridor_update_ids` / `equation_update_ids` in `eco_shards::response_turn`

The ResponseTurn design uses **JSON fragments** for flexible, cross-plane metadata (e.g., `rx_map_json`), and the stack explicitly recommends treating such fields as JSON values at storage and parsing time. For `rxmapjson`, the pattern is: store an opaque JSON blob, parse it as `serde_json::Value` and project into typed maps in plane-specific crates.[70]

By the same pattern, `corridor_update_ids` and `equation_update_ids` are intended as **JSON arrays of strings** (e.g., `["EcoCoreParametersPhoenix2026v3","CorridorBand-rt90-v2"]`), not comma-delimited strings.[file:13] This keeps them:

- Unambiguous (no escaping issues versus CSV-style lists).
- Directly usable by tools that already treat ResponseTurn's JSON fields as `serde_json::Value` and then materialize `Vec<String>` when needed.[71] [70]

Where these IDs are mirrored into CSV, the recommended pattern is to keep the JSON in a single column (e.g., a JSON-encoded array string) rather than inventing an ad-hoc comma-separated syntax.[71]

83. Shard-level deduplication service

Your shard model already assumes **append-only, immutable RFC 4180 CSVs with evidence hashes**, which fits deduplication very well. A deduplication service can be built as a non-actuating, index-sidecar:[69] [70]

1. **Canonical hash:** use the existing `evidencehex` / ProvenanceKernel definition: compute a deterministic hash over the canonical serialization of all mutable shard fields, excluding `evidencehex`, signatures, and ledger TX fields. This already exists as your shard-integrity primitive.[69] [70]
2. **Index family:** maintain a small qpudatashard family, e.g., `ShardDedupIndex2026v1.csv`, with rows:

   `hash, shard_family, logical_path, lane, size_bytes, first_seen_at, n_duplicates`.[70]

   A scanner walks `mntoss/shards`, recomputes the hash if missing, and records entries.
3. **Dedup decisions:** when multiple shards share the same hash:
   - Mark one path as the **canonical** location (e.g., first in time, or under the spec'd directory layout).[70]
   - For additional copies, either:
     - Replace them with **filesystem hard links** to the canonical file, if the backend supports it and your governance allows (on POSIX-like local volumes).[70]
     - Or, keep them as tiny "pointer shards" that record a reference to the canonical path, while maintaining the same schema and governance metadata.[69] [70]
4. **Governance:** dedup is non-semantic; the hash includes all physics, KER, and corridor references, so deduplicated shards are **byte-identical**. Promotion and KER decisions still see the same content; only storage layout changes. CI and ALN invariants treat pointer shards as valid only if they resolve to a canonical shard with matching `evidencehex`.[69] [70]

This service has no actuation authority; it only moves or links shard files and writes index shards, keeping it safely within your non-representability and provenance guarantees.[69] [70]

84. Crates with `unsafe` for FFI / hardware

Existing policy: **all core ecosafety math, ALN, governance, and agent-interface crates are `#![forbid(unsafe_code)]` and must remain pure safe Rust.** Unsafe is allowed only in small, leaf IO / VHAL crates (e.g., memory-mapped IO, SIMD, FFI) with extra governance and testing.[70]

For hardware FFI crates:

- They should be **separate crates**, e.g., `eco_node_vhal`, not part of `ecosafety_core`, `eco_shards`, or `kercore`.[69] [70]
- Mark them explicitly with a crate-level comment and ALN tag such as `plane = "vhal"`, `safety_role = "IO-unsafe"`, and document their minimal FFI surface.[71] [69]

- Keep `unsafe` blocks tightly scoped around FFI bindings and pointer manipulation; expose only **narrow, safe APIs** that hand back normalized `RiskCoord`, `MetricFields`, `CorridorDecision`, etc., never raw voltages or unconstrained setpoints.[70] [69]

Governance requirements for such crates:[71] [69] [70]

- Dedicated fuzzing and property-based tests, especially around boundary conditions and FFI invariants.
- CI rules ensuring no crate tagged `safety_spine` (ecosafety core, ALN, KER math, NodePlacement, agent interface) depends on any `unsafe` crate.
- Shard-anchored specs (`SPEC-FFI-SAFETY.aln`) describing allowed FFI use, audited and DID-signed.

So the rule is: **keep `#![forbid(unsafe_code)]` in the safety spine; concentrate unavoidable `unsafe` in clearly labeled, governance-heavy IO/FFI crates that sit at the edges.**[69] [70]

85. Ecosafety audit trail retention policy

Your docs already define a reserved **10 TB capacity specifically for error codes and audit trails** and sketch formats and retention. Within your grammar this is a dedicated shard family (or families) like `DecisionLogEntry`, `GuardAudit`, `CIOutcome`, and related qpudatashards plus compact summaries.[70] [69]

Format:[71] [70]

- Primary records as RFC 4180 CSV qpudatashards: each row carries `id`, `timestamp`, `location_hex`, `corridor_id`, K/E/R, full r_x vector, residual V_t, lane, `kerdeployable`, plus `evidencehex` and signing DIDs.
- Optional higher-level JSON/ALN summary shards for compressed views (e.g., KER summaries per run or per corridor set).

Retention:

- **Minimum 10 years** for KER-critical events: corridor changes, ecosafety breaches, neurorights-relevant decisions, major lane promotions, and deploy/rollback decisions.[70]
- The "10 TB reserved" is a **floor that cannot be repurposed** for non-audit data without a new, quorumsigned governance shard explicitly changing the policy.[70]
- Older, non-critical logs may be compacted into summaries, but the critical decision and corridor-change shards themselves must remain available and verifiable for at least that 10-year window.[69] [70]

Indexing is via small shard-native indices (by corridor, asset, region, time) so reconstructing any decision trail requires scanning only a modest number of files, not the full archive. All of this remains non-actuating and is anchored with `evidencehex` and DIDs, enabling replayable proofs of safestep and kerdeployable over historical windows.[71] [69] [70]

KER for this answer

- K ≈ 0.95 — All definitions and designs are directly grounded in your existing lane semantics, KER bands, FFI policy, dedup, and audit-trail descriptions.[71] [69] [70]
- E ≈ 0.91 — Clarified lane promotion, dedup, unsafe governance, and retention policies increase eco-governance reliability and transparency.
- R ≈ 0.12 — Remaining risk lies in corridor calibration, dedup tooling correctness, and governance enforcement, all explicitly surfaced and bounded by existing shards, KER gates, and DIDs.[71] [69] [70]

⁂

86. DQ – In `ecosafety_core::residual`, is `Vt` computed as a simple sum or a more complex aggregation (e.g., moving average) for windowed KER?

87. RQ – What is the best practice for monitoring `/mnt/oss` disk usage and alerting when free space falls below a threshold, given the 16E capacity?
88. OI – The `eco-ci-validate` binary exits with code 2 on failure; how does this integrate with `git bisect` or other developer tools that expect standard exit codes?
89. DR – Provide a definition for "shard index" as referenced in `/mnt/oss/index/`—what exact information is stored per shard, and how is the index updated?
90. DQ – In `oss_vfs::open_shard()`, is the file path validated against the expected shard family and lane based on the path structure, or is any file under `/mnt/oss` readable?

Answering 86–90 in order.

86. `ecosafety_core::residual` — how `Vt` is computed

In `ecosafety_core::residual`, V_t is always computed as a **simple weighted sum of squared risk coordinates** $V_t = \sum_j w_j r_{j,t}^2$ for a single step, not as a moving average. Windowed KER is built on top of this by keeping a separate `KerWindow` / **KER summary** that aggregates per-step decisions (fraction of Lyapunov-safe steps, max risk coordinate, etc.), but the residual kernel itself is strictly instantaneous.[72] [73]

So: compute V_t for each step from the current `RiskVector` and `LyapunovWeights`, then K, E, R over a window are derived by counting/aggregating those per-step results in `KerWindow`, not by smoothing V_t itself.[73] [72]

87. Monitoring `/mnt/oss` disk usage at 16E

Best practice is to treat `/mnt/oss` as a **governed storage surface** with shard-aware metrics plus standard OS monitoring, rather than relying only on raw `df`.[72] [73]

At minimum:

- Run an OS-level monitor (e.g., `node_exporter` + Prometheus, or equivalent) to track total and free bytes on `/mnt/oss` and its backing volumes; set alerts when **free space drops below conservative thresholds** (e.g., 20%, then 10%).[72]
- Add a **shard-aware usage index** under `/mnt/oss/index/` that maintains per-prefix and per-family byte counts (e.g., bytes under `/mnt/oss/shards/region=Phoenix/lane=PROD`), updated by CI or a small Rust daemon, so you can alert on runaway growth by lane, region, or family before hitting global limits.[73] [72]
- Emit **metrics shards** (e.g., `StorageMetricsYYYYMMDD.csv`) that log periodic usage snapshots, KER for storage risk (e.g., `rstorage`, `rheat`), and eco-impact of storage; these become part of your audit trail and can drive governance actions like shard compaction, dedup, or corridor tightening for storage-heavy workloads.[72]

Given the notional 16E capacity, alerts should be **based on absolute free space and trend**, not just percentage, but the semantics remain the same: guardrails in CI and operations so ecosafety shards and indices never exhaust storage in ways that would silently drop data.[73] [72]

88. `eco-ci-validate` exit code 2 and `git bisect`

The stack assumes **non-zero exit means failure**, without overloading specific meanings beyond "success vs failure," which keeps it compatible with `git bisect`, `pre-commit` hooks, and CI systems that only care about zero/non-zero. `eco-ci-validate` using exit code `2` on failure still works with `git bisect` because:[72]

- `git bisect run` treats **exit code 0 as "good" and any non-zero as "bad"**, regardless of the specific code.
- Your scripts already pattern on this: ecosafety modes call `eco-ci-validate` and propagate its exit status, so bisect and other tools will simply see "bad" when ecosafety invariants fail, without needing to special-case `2`.[72]

If desired, you can standardize on `1` for generic failure and reserve `2` for "ecosafety violation" in documentation, but bisect integration itself is already sound because it only depends on zero vs non-zero.[72]

89. Definition of "shard index" under `/mnt/oss/index/`

A **shard index** is an auxiliary qpudatashard family that stores **small, query-optimized metadata rows per data shard**, enabling fast lookups and range scans without walking the full `/mnt/oss/shards` tree.[73] [72]

Per-shard index rows typically include:[73] [72]

- Shard identity: logical shard ID or path, family ID, region/basin, lane, ALN spec ID, and `evidencehex`.
- Size and layout: size in bytes, row count, date/partition key, storage class or backend.
- Risk summary: KER triad (K, E, R) for that shard or a recent window, max risk coordinate, and flags like `kerdeployable` or `BlockedByCalib`.
- Routing hooks: node IDs, corridor IDs, and pointers (URIs or relative paths) into `/mnt/oss/shards/...` so higher-level services can find the shard quickly.

Update strategy:[72]

- Index shards are **append-only**; you never rewrite index rows in place.
- On shard creation or promotion, CI or a small indexer service writes a new row into the appropriate index shard (e.g., `IndexPhoenix2026v1.csv`) with the metadata above.
- Periodic compaction can create higher-level summary indices (e.g., per-day / per-region aggregates) but always as new shards, preserving full history for audit.

This matches the recommended exabyte-scale pattern: **queries hit `/mnt/oss/index/` first**, which is small and cache-friendly, and only then follow pointers to large data shards under `/mnt/oss/shards/` when necessary.[73] [72]

90. `oss_vfs::open_shard()` path validation

`oss_vfs::open_shard()` is **not a blind "read anything under `/mnt/oss`" API**; it is specified to be **shard-aware and governed by VFS snapshots and lane/family rules.**[73] [72]

The pattern is:

- Allowed logical paths and operations are defined by **VFS snapshot shards** (cc-vfs manifests) that specify which directories and shard families a given tool or crate may access and with which lanes.[72]
- `open_shard()` takes a logical shard path and **validates it against these manifests**, including:
  - The path structure (e.g., `/mnt/oss/shards/family=X/lane=PROD/…`) must match a known shard family and lane pattern.
  - The caller's lane / governance profile must allow access to that family and lane.[73] [72]
- Only if the path and lane/family mapping are valid does it resolve to the underlying POSIX or object-store path and open the file; otherwise it fails (error / denial), even if a file physically exists at that path.[72]

So **not every file under `/mnt/oss` is readable via `oss_vfs::open_shard()`**; readability is constrained by the shard family definitions, lane rules, and the predeclared VFS snapshot manifests, in line with your "no schema, no write / no corridor, no access" philosophy.[73] [72]

Turn-level KER

- K ≈ 0.95 — Answers reuse your existing residual/`KerWindow`, shard indexing, VFS, and CI semantics.[73] [72]
- E ≈ 0.91 — Clarifying residual math, index content, VFS gating, and monitoring improves ecosafety and auditability.
- R ≈ 0.12 — Residual risk lies in operational tuning (disk thresholds, index freshness, VFS manifests), which are explicitly exposed and governable via shards and CI.[72] [73]

⁂

91. RQ – How can we implement a "shard-based canary deployment" where new corridor bands are first applied to a small percentage of traffic and only rolled out if R does not increase?

92. **OI** – The `agent_interface` vocabulary includes `ProposePatch`; does this cover changes to ALN spec files, or is there a separate action for spec changes?
93. **DR** – Define "ecosafety kernel version"—how are changes to the normalization and residual kernels (e.g., from sum of squares to max) versioned and rolled out?
94. **DQ** – In `eco_shards::eco_core_parameters`, how are corridor rows with duplicate `varid` for the same region handled? Is it an error, or does the latest version take precedence?
95. **RQ** – Can we implement a "shard-based IDE plugin" that displays KER badges next to code files, showing the ecosafety impact of the code based on associated shards?

Answers 91–95 inline, with brief designs and how they fit the existing grammar.

91. Shard-based canary deployment for new corridors

You can implement shard-based canaries by treating a new corridor set as a separate, lane-scoped spec and rolling it out on a governed subset of shards, with promotion based on KER and residual behavior. The pattern already appears in your "corridor tightening experiment" description: propose new bands, replay historical shards, and accept only if R does not increase and K/E stay within thresholds.[75]

Operationally, you define a new `EcoCoreParameters…vX+1` / `corridors.ecosafety.vX+1` ALN spec and bind it to a small fraction of traffic via a `NodePlacement`-style shard: a subset of nodes or corridors get the new spec while the rest stay on vX. Each canaried node emits KER windows tagged with its corridor version; CI then compares K, E, R and V_t trajectories between vX and vX+1 over matched time windows, and only promotes vX+1 to the main PROD lane if R is non-increasing (and typically strictly lower) at the same or higher knowledge and eco-impact. That gives you a canary where "percentage of traffic" is literally "fraction of shards/nodes whose `NodePlacement` points at the new corridor set," fully shard-auditable.[76] [75]

92. `agent_interface::ProposePatch` and ALN spec changes

The corpus makes a clear distinction between regular code/data edits and ALN contract evolution, and treats ALN specs as governing law rather than normal artifacts. `ProposePatch` is described as a generic, shard-scoped change proposal (code or data), but ALN spec changes carry additional invariants like "tightest band wins," explicit version tags, and proof hexstamps.[76]

Because of that, best practice is to reserve a separate, more constrained verb for spec changes (for example, a `ProposeSpecChange` or "contract upgrade" action) that always results in dedicated "spec migration" and "corridor tightening" shards, not just a normal diff. The macro work you already sketched (`deriveAlnShard` with `alncontractversion` and "no mixed major versions") assumes spec evolution is rare and heavily gated, so letting general `ProposePatch` mutate `.aln` directly would bypass those governance expectations. In other words: `ProposePatch` is fine for Rust, docs, and ordinary shards, but ALN specs should go through a stricter, spec-specific proposal path.[76]

93. Definition of "ecosafety kernel version"

An "ecosafety kernel version" is effectively the versioned tuple of your normalization and residual semantics: the corridor normalization function family, the weight vector, the residual form V_t, and the KER decision rules pinned to a specific ALN contract. Today that kernel is defined as piecewise-affine corridor normalization (safe/gold/hard bands) feeding a Lyapunov residual $V_t = \sum_j w_j r_j^2$ with non-increase and hard-band invariants, plus KER thresholds and lane rules.[75] [76]

Changes like "sum of squares vs max" or different normalization curves would constitute a new kernel version and must be encoded in ALN (e.g., `EcosafetyKernel2026v2` vs `…v3`, with `normalizationkernel` and residual type fields), not just code. Rollout then follows the same pattern as corridor tightening: introduce the new kernel as a separate spec, replay historical shards and atlas windows through both kernels, and only allow promotion if decisions (safestep, kerdeployable) are equivalent or strictly more conservative and K/E remain above deployment thresholds. The macro and FFI designs already assume explicit kernel IDs and "tightest band wins," so kernel upgrades become governed, shard-auditable events rather than silent code changes.[75] [76]

94. Duplicate `varid` rows in `eco_core_parameters`

Your existing rules around corridors are "no corridor, no build" and "corridor tables must be complete and signed before a shard is valid," and the type-level `CorridorBands` builder pattern encodes "exactly one complete band per variable." That strongly implies that duplicate `varid` rows for the same region in `eco_shards::eco_core_parameters` should be treated as a structural error, not as "latest wins."[75] [76]

At the ALN and macro level, you already propose failing compilation when mandatory columns are missing or duplicated, and enforcing a single canonical band per `varid` and

contract version. Mirroring that for runtime CSV load means: when the loader sees multiple corridor rows for the same `varid` and region under a single kernel/spec, it must reject the parameters shard (and thus fail CI) rather than pick one implicitly. If you need multiple candidate corridors for the same physical variable, they should appear as explicitly versioned or lane-scoped specs, not as unlabelled duplicates in `EcoCoreParameters`.[76] [75]

95. Shard-based IDE plugin with KER badges

A shard-based IDE plugin that decorates files with KER badges is not only feasible but directly aligned with your "KER latency" and "knowledge-factor aggregates" work. You already describe precomputing KER aggregates per shard family and topic into small `KerAggregateShard…` CSVs so that queries like "topic KER trend in the last 30 days" can be served in milliseconds.[75]

The plugin can run purely as a non-actuating client of those shards: for each code file, it looks up its associated shard families or assets via a small mapping shard, then reads precomputed K/E/R or lane status from the aggregates and displays a badge (for example "K=0.94, E=0.91, R=0.13, lane=PROD"). All heavy computation remains in CI and background shard emitters; the IDE just renders shard state, staying within your rules that non-Rust tooling must not change corridors or KER semantics and should treat shards as immutable evidence.[76] [75]

⁂

96. OI – The placeholder script generates 120 crates; if the coder wants only a subset (e.g., 10), is there a parameter to control the count, or must they manually delete the extras?

97. **DR** – Provide a definition for "risk coordinate direction"—how is the `direction` field in `CorridorBands` used in normalization, and what are the allowed values?
98. **DQ** – In `eco-ci-validate`, how are shard validation errors aggregated? Is there a limit on the number of errors reported before the tool exits?
99. **RQ** – How should we design a "shard backup and restore" procedure that ensures all `evidencehex` and DIDs remain verifiable after restoration to a new `/mnt/oss` instance?
100. **OI** – The current documentation assumes coders are familiar with ecosafety grammar and KER; what is the onboarding plan for new contributors, and where are the learning materials stored as shards?

Here are direct answers for 96–100, aligned with the existing ecosafety and shard grammar, plus per-item K/E/R scores.

96. Placeholder script crate count control (OI)

In the current design, the 120-crate placeholder range is treated as scaffolding, not as a fixed requirement, and the governance guidance is to avoid scaling this pattern directly to thousands of crates because of workspace bloat and Cargo parsing overhead. The recommended mitigation is to move from "one crate per conceptual shard" to logical shards inside a smaller set of crates, and to enforce workspace-size corridors in CI rather than rely on manual deletion.[77]

Concretely, the docs do not define a first-class "crate_count" parameter on the PowerShell generator you have today; instead they prescribe either: (a) refactor the generator to use a loop over a configurable range, or (b) retire the placeholder crates by removing them from `workspace.members` and CI, then deleting them, following the "removing placeholder crates safely" steps. Those steps explicitly say: update CI so it no longer assumes those crates exist, prune `Cargo.toml` membership and ALN references, run ecosafety checks, then delete, which shows that reduction in count is expected to be scripted and governed, not hand-delete only.[77]

Knowledge factor K ≈ 0.93 (directly grounded in the placeholder and workspace-bloat discussion). **Eco-impact E ≈ 0.90** (keeping the workspace small reduces wasted compute and energy). **Risk-of-harm R ≈ 0.13** (badly managed placeholders hurt coder throughput but not direct ecological safety).[77]

97. "Risk coordinate direction" in CorridorBands (DR)

CorridorBands already encode corridors as the envelope that maps physical metrics into normalized risk coordinates $r_j \in [0, 1]$, and the schema mentions fields like `varid`, units, safe/gold/hard bounds, weight, and `lyapchannel`. In the corridor definition text, "direction" is treated conceptually as whether higher physical values are worse or better, because normalization uses piecewise-linear kernels anchored at safe/gold/hard bands: for classical pollutants (PFAS, E. coli, WBGT) higher is worse, whereas for some "higher-is-good" metrics (e.g., hydraulic buffering capacity) the mapping is inverted.[78] [77]

In the excerpts you provided, `direction` is not yet formalized as a separate enumerated column in the CSV schema, but its semantics are implicit in the normalization kernel and corridor bands. The allowed values in practice are therefore a two-way choice: "higher-is-worse" (monotone increasing risk with the variable) or "higher-is-better" (monotone decreasing risk with the variable), both clamped into $0, 1$ and used identically by the residual kernel $V_t = \sum_j w_j r_j^2$. The research thread explicitly calls out formalizing this as a typed coordinate (e.g., "RiskCoord01" with direction) in EcoNetSchemaShard, but the current implementation uses the same Lyapunov kernel for all directions and keeps the sign convention in the corridor rows, not in a separate field.[78] [77]

K ≈ 0.90 (direction is described implicitly but not pinned to a named enum in the snippet). **E ≈ 0.91** (making direction explicit will improve correctness of normalization across materials and hydrology). **R ≈ 0.13** (mis-set direction would mis-score risk, which is why corridor calibration is surfaced as a high-priority research action).[78] [77]

98. eco-ci-validate error aggregation (DQ)

The eco-ci-validate design you have described emphasizes incremental validation and bounded parallelism, but the excerpts do not spell out a hard numeric cap on the number of errors emitted before exit. eco-ci-validate is meant to consume a set of shards (or a delta shard) and run ALN schema checks, corridor presence checks, Lyapunov residual and KER gates, producing a KER summary shard or JSON plus an error report that CI treats as a hard failure if any shard violates invariants.[77]

There is substantial design detail about how to keep validation scalable: using a "validation index" shard to avoid re-validating unchanged shards, parallelizing across shard families with bounded concurrency (via rayon or async semaphores), and writing results into a small number of summary artifacts. But nowhere in the provided text is there a specification like "stop after N errors" or "aggregate at most N per shard"; instead the model is "treat any schema or corridor break as a hard failure for that shard family and fail CI." So the current spec is: errors are aggregated at the shard-validation level and surfaced as CI failure, and any additional limits (e.g., truncating per-file error lists) remain an open, implementer-choice detail that you may want to shard and specify.[77]

K ≈ 0.88 (we know the aggregation pattern, but not an explicit error limit constant). E ≈ 0.90 (incremental index and bounded parallelism keep validation eco-efficient at scale). R ≈ 0.15 (too-verbose error output could hurt ergonomics, but it does not change deployability decisions, which still gate on any non-empty failure set).[77]

99. Shard backup and restore with verifiable evidencehex/DIDs (RQ)

Your ProvenanceKernel and shard governance docs already define evidencehex as a canonical hash over shard content (excluding evidence/signature fields themselves) plus DID-anchored signatures, precisely so that shards can be moved, backed up, and replayed without losing verifiability. The safe procedure for backup and restore to a new `/mnt/oss` instance follows from that:[78] [77]

First, you snapshot shards at the file level without modification, preserving RFC4180 structure, metarows (ALNSHARDID, ALNSPECHASH, KERCONTEXT, lane, etc.), and all data rows; no in-place edits are allowed, and any migration must produce new shards with new evidencehex instead of mutating existing files. Second, you restore to a new `/mnt/oss` that is governed by the same VFS routing shards and EcoNetSchemaShard specs, so logical paths and lane/topic layout remain consistent; the design already anticipates using VFS map shards to abstract physical volumes, which means a new instance can be wired under the same logical root without changing shard contents.[78] [77]

After restoration, you verify integrity by running the evidencehex core kernel over each shard on the new `/mnt/oss`, recomputing the canonical digest from normalized bytes (metarows, headers, data rows) and checking equality with the stored evidencehex; you also replay DID signatures (signinghex) against the known Bostrom keys for your DIDs and optionally cross-check parentevidencehex chains and any ledger transaction hashes. Because evidencehex deliberately excludes its own field and signature fields from the hash,

this recomputation is well-defined and will detect any corruption or silent modification during backup/restore; any mismatch forces the shard into a high-risk `rcalib` band and out of PROD lanes until resolved.[78] [77]

K ≈ 0.95 (directly follows the ProvenanceKernel and evidencehex design). E ≈ 0.91 (robust, replayable provenance supports long-term eco-governance and audit). R ≈ 0.12 (main risk is backup tooling mishandling files, which this procedure surfaces via hash and DID checks).[77] [78]

100. Onboarding plan and shard-stored learning materials (OI)

The corpus repeatedly assumes contributors understand ecosafety grammar (corridors, residuals, KER, lanes) and shard practices, and it identifies this assumption as a gap that should be closed with shard-backed learning materials, but the snippets you shared do not yet define a finalized "onboarding shard family." However, several documents outline a pattern of using shard families like EcoNetSchemaShard, ResearchPlanShard, and "what-can-be-researched-to-help-..." to encode documentation, research actions, and spec guidance as first-class shards under `/mnt/oss`, with ALNSPECHASH and evidencehex for each.[78] [77]

Within that pattern, the intended onboarding plan is: create dedicated documentation shard families (for example, EcoSafetyOnboarding2026v1, KERPrimerShard2026v1, CorridorTutorialShard2026v1) that contain short, schema-governed rows describing concepts, examples, and links to ALN specs and Rust crates, and store them in the spec/knowledge area of `/mnt/oss` so they become part of the same AI and CI knowledge graph used for code and corridor decisions. The AI knowledge-graph description confirms that ALN specs, corridor tables, and shards form a federated read-only index for agents; extending that to onboarding shards means new contributors see the same governed artifacts as the rest of the system, and their training materials can themselves be versioned, hex-stamped, and KER-scored for knowledge, eco-impact, and risk.[77]

K ≈ 0.90 (the pattern for spec-backed learning shards is explicit, but the exact onboarding family names are not yet fixed). E ≈ 0.92 (good onboarding directly accelerates ecosafety work and reduces accidental misuse). R ≈ 0.13 (poor or missing onboarding is a human-factor risk, which this plan addresses by treating learning materials as governed shards).[77]

⁂

1. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
2. daily-rust-and-aln-code-genera-g0Rz_p5bTGCq6sEaIODFtg.md
3. this-research-focuses-on-compl-Hu72iH.HT7q0T5G8LeHUwQ.md
4. chat-export-1774694199428.json
5. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
6. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
7. chat-export-1774694199428.json

8. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
9. this-research-focuses-on-estab-r5QFi0s1R32uvbG7t9bn7Q.md
10. identify-missing-files-search-X2wjuVcDQBCMOLyHNKa5uA.md
11. this-research-focuses-on-impro-XUZXv4KCT_6HSQ2689fyNw.md
12. hydrological-buffering-researc-VvmYz1SfQzSmU6gdvtzmFQ.md
13. produce-codes-files-documents-XXktQZ69TIK3V7e.569auw.md
14. this-research-focuses-on-creat-VXQvqLhcRKG2u8UIakEr3Q.md
15. this-research-focuses-on-lower-B8mu0wUvRmSiPNT8_FFLFw.md
16. this-research-focuses-on-lower-B8mu0wUvRmSiPNT8_FFLFw.md
17. this-research-focuses-on-lower-B8mu0wUvRmSiPNT8_FFLFw.md
18. this-research-focuses-on-advan-m_vs_yJaSFOD.EEjdWyGng.md
19. this-research-investigates-the-qbNorDiTTZy3DLf7mxcpdw.md
20. this-research-focuses-on-creat-qWLKDItzRCCKWxAp9M8BOg.md
21. what-can-be-researched-to-help-edcPb6nLQDOGvQPyZshVGQ.md
22. this-research-focuses-on-three-LR_Dl6L5Su6s.LwhVMaUfg.md
23. this-research-focuses-on-three-E2_0armHRY2k.jdJeLAGLA.md
24. this-research-focuses-on-advan-wOo689VxTbqIt27cJVUFeQ.md
25. this-research-focuses-on-empir-V74CETA4Rwim8mOXcjGVDg.md
26. this-research-focuses-on-evolv-DmFTMoizQ5uug0CQsTtV1g.md
27. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
28. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
29. chat-export-1774694199428.json
30. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
31. chat-export-1774694199428.json
32. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
33. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
34. chat-export-1774694199428.json
35. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
36. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
37. chat-export-1774694199428.json
38. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
39. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
40. chat-export-1774694199428.json
41. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
42. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
43. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md

44. chat-export-1774694199428.json
45. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
46. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
47. chat-export-1774694199428.json
48. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
49. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
50. chat-export-1774694199428.json
51. chat-export-1774694199428.json
52. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
53. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
54. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
55. chat-export-1774694199428.json
56. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
57. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
58. chat-export-1774694199428.json
59. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
60. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
61. chat-export-1774694199428.json
62. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
63. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
64. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
65. chat-export-1774694199428.json
66. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
67. chat-export-1774694199428.json
68. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
69. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
70. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
71. chat-export-1774694199428.json
72. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
73. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
74. chat-export-1774694199428.json
75. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
76. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md
77. this-document-provides-exactly-3WKgRVZ9Tv.H44433auPkQ.md
78. this-research-focuses-on-stren-zrdP8rD6SOK7BD3VwRNRzQ.md